

Министерство образования Республики Беларусь
Учреждение образования «Белорусский государственный университет
информатики и радиоэлектроники»

Факультет компьютерных систем и сетей

Кафедра электронных вычислительных машин

Дисциплина: СиФОВМ

ПОЯСНИТЕЛЬНАЯ ЗАПИСКА
к курсовому проекту
на тему

РАЗРАБОТКА МИКРО-ЭВМ НА ПЛИС

БГУИР КП 1-40 02 01 029 ПЗ

Студент: Мигулин Д. Н., гр.250541

Руководитель: Воронов А. А.

Минск 2026

Вариант № 8

Тип архитектуры	Принстонская
Разрядность шины адреса	16 бит
Разрядность шины данных	8 бит
ПЗУ	Асинхронное
ОЗУ	Асинхронное
Тип адресации	Прямая, прямая регистровая
Количество регистров общего назначения (РОН)	8 шт.
Арифметические команды АЛУ	SUB (вычитание)
Логические команды АЛУ	NOTZ (инверсия по флагу Z), AND (логическое умножение)
Сдвиговые команды АЛУ	SRL (логический сдвиг вправо)
Тип арбитража шин	Централизованный последовательный
Организация кэш-памяти	k=4 (4-канальная), алгоритм замещения LRU, сквозная запись без отображения
Стек	Объём 11 байт, направление роста – вниз
Схема предсказания переходов	A5, 4-битный GHR (Global History Register)
КПДП (контроллер прямого доступа к памяти)	Начальный адрес = 10, объём передачи = 6 байт
Обязательные команды	MOV adr, reg; MOV reg, adr; JMP adr; PUSH reg; POP reg; HLT

СОДЕРЖАНИЕ

ВВЕДЕНИЕ.....	4
1 Разработка общей структуры микро-эвм.....	5
1.1 Функциональный состав микро-ЭВМ	5
1.1.1 Блок запоминающего устройства	5
1.1.2 Блок устройства управления (УУ).....	5
1.1.3 Блок регистров общего назначения	6
1.1.4 Блок общих операций	6
1.1.5 Блок арифметико-логического устройства (АЛУ).....	6
1.1.6 Блок стека	7
1.1.7 Блок кэша	7
1.1.8 Блок контроллера прямого доступа к памяти (DMA)	7
1.1.9 Специальные регистры микро-ЭВМ	7
1.2 Разработка системы команд.....	8
1.3 Описание взаимодействия всех блоков микро-ЭВМ при выполнении команд программы.....	10
1.3.1 Исполнительные блоки:.....	10
1.3.2 Взаимодействие с памятью и кэшем	11
1.3.3 Взаимодействие с регистрами общего назначения.....	11
1.3.4 Предсказание условных переходов	12
2 Разработка основных устройств микро-эвм.....	13
2.1 Запоминающие устройства	13
2.2 Устройство управления.....	14
2.2.1 Центральный узел распределения управляющих сигналов....	14
2.2.2 Узел местного управления фазами выполнения команд.....	15
2.2.3 Блок указателя команд IP и блок регистра кода команд IR....	16
2.2.4 Блок управления стековой памятью	18
2.3 Арифметико-логическое устройство	19
2.3.1 Логическое построение АЛУ и блок РОНов	19
2.3.2 Реализация арифметических и логических операций	21
2.4 Организация кэш-памяти процессора.....	24
2.5 Описание системы предсказания переходов	26
2.6 Аппаратура и функционирование КПДП.....	28
2.7 Описание арбитра шины	31
3 Функциональное моделирование	33
3.1 Функциональное моделирование блока ОЗУ	33
3.2 Функциональное моделирование центрального узла распределения управляющих сигналов.	34
3.3 Функциональное моделирование узла местного управления фазами выполнения команд	36
3.4 Функциональное моделирование блока указателя команд IP и блока регистра кода команд IR.....	38
3.5 Функциональное моделирование блока управления стековой памятью.....	38

3.6 Функциональное моделирование блока РОНов	39
3.7 Функциональное моделирование блока арифметических и логических операций	40
3.8 Функциональное моделирование блока кэш-памяти.....	42
3.9 Функциональное моделирование блока предсказания переходов.....	43
3.10 Функциональное моделирование блока блока КПДП.....	45
3.11 Функциональное моделирование блока арбитража шины.....	46
3.12 Описание результатов функционального моделирования работы микро-ЭВМ.....	47
ЗАКЛЮЧЕНИЕ	51
ПРИЛОЖЕНИЕ А (обязательное) Структурная схема блока памяти...	53
ПРИЛОЖЕНИЕ Б (обязательное) Структурная схема блока центрального узла распределения управляющих сигналов.....	54
ПРИЛОЖЕНИЕ В (обязательное) Структурная схема блока узла местного управления фазами выполнения команд.....	55
ПРИЛОЖЕНИЕ Г (обязательное) Структурная схема блока указателя команд IP и блока регистра кода команд IR.....	56
ПРИЛОЖЕНИЕ Д (обязательное) Структурная схема блока управления стековой памятью	57
ПРИЛОЖЕНИЕ Е (обязательное) Структурная схема блока регистров общего назначения	58
ПРИЛОЖЕНИЕ Ж (обязательное) Структурная схема блока АЛУ	59
ПРИЛОЖЕНИЕ И (обязательное) Структурная схема блока кэш- памяти	60
ПРИЛОЖЕНИЕ К (обязательное) Структурная схема блока предсказания переходов	61
ПРИЛОЖЕНИЕ Л (обязательное) Структурная схема блока КПДП....	62
ПРИЛОЖЕНИЕ М (обязательное) Структурная схема блока арбитража шины	63
ПРИЛОЖЕНИЕ Н (обязательное) Структурная схема блока микро- ЭВМ	64
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	65

ВВЕДЕНИЕ

Разрабатываемая микро-ЭВМ основана на принстонской архитектуре, что означает использование общей шины для программ и данных. Данный подход упрощает организацию памяти, но требует внимательного учёта при проектировании операций, так как обращения к ПЗУ и ОЗУ выполняются через единый канал.

Аппаратная часть включает адресную шину разрядностью 16 бит и шину данных разрядностью 8 бит. Такое сочетание позволяет адресовать до 64 Кбайт памяти при сравнительно небольшой ширине слова. Память представлена асинхронным ПЗУ и ОЗУ, что упрощает реализацию, но накладывает ограничения на скорость доступа. Тип адресации – прямая регистрация, что обеспечивает базовый набор способов обращения к операндам.

В системе предусмотрено восемь регистров общего назначения, которые будут использоваться для хранения промежуточных данных и организации вычислений. Арифметико-логическое устройство (АЛУ) реализует команды варианта: SUB (вычитание), NOTZ (инверсия по флагу нуля), AND (логическое умножение), SRL (логический сдвиг вправо).

Для повышения производительности вводится кэш-память с множественно-ассоциативным отображением ($k = 4$). Алгоритм замещения строк выбран классический – LRU (Least Recently Used), что обеспечивает более эффективное использование кэша по сравнению с простыми методами. Синхронизация кэша с основной памятью реализуется по схеме «сквозная без отображения» (Write-through without allocation), что упрощает проектирование, но требует аккуратного управления потоками данных.

Для оптимизации выполнения команд условных переходов реализована схема динамического предсказания на основе автомата A5 с 4-битным регистром глобальной истории (GHR).

Доступ к общей шине координируется через централизованный последовательный арбитраж. В системе также предусмотрен контроллер прямого доступа к памяти (КПДП) с начальным адресом 10 и объёмом передачи 6 байт. Стековая память организована на 11 ячеек с направлением роста вниз.

Набор обязательных команд включает MOV adr, reg; MOV reg, adr; JMP adr; PUSH reg; POP reg; HLT, а также прямую адресацию памяти. При разработке арифметико-логического устройства и устройства управления предусмотрено наличие регистра флагов, который будет использоваться для реализации условных переходов и анализа результатов операций.

1 РАЗРАБОТКА ОБЩЕЙ СТРУКТУРЫ МИКРО-ЭВМ

1.1 Функциональный состав микро-ЭВМ

Архитектура микро-ЭВМ построена на принстонской модели, что означает использование общей шины для программ и данных. Это решение упрощает организацию памяти, но требует внимательного проектирования операций, так как обращения к ПЗУ и ОЗУ выполняются через единый канал. В состав микро-ЭВМ входят несколько функциональных блоков, каждый из которых выполняет строго определённые задачи и взаимодействует с другими через систему шин (адресную, данных и управляющую).

1.1.1 Блок запоминающего устройства

Основой вычислительной системы является память, представленная двумя типами:

- ROM (Read-Only Memory) – асинхронное постоянное запоминающее устройство, предназначенное для хранения программного кода и неизменяемых данных.
- RAM (Random Access Memory) – асинхронное оперативное запоминающее устройство, используемое для хранения промежуточных данных и результатов вычислений.

Управление обращениями к памяти осуществляется специальной схемой, которая координирует работу с общей шиной. Принстонская архитектура предполагает, что ROM и RAM используют один и тот же канал, поэтому устройство управления должно обеспечивать корректное разделение обращений и предотвращать конфликты.

1.1.2 Блок устройства управления (УУ)

Устройство управления является «мозгом» микро-ЭВМ. Оно отвечает за выборку команд из памяти, их декодирование и формирование управляющих сигналов для всех остальных блоков. В состав УУ входят:

- блок специальных регистров (IP, IR, AR, DR, FR), которые обеспечивают хранение служебной информации;
- схема декодирования команд, преобразующая машинный код в управляющие сигналы;
- схема контроля итераций операций, необходимая для выполнения сложных инструкций;
- схема синхронизации работы с кэшем, которая обеспечивает согласованность данных между памятью и кэш-блоком;
- схема контроля переполнения регистра IP, предотвращающая выход за пределы адресного пространства.

- схема динамического предсказания условных переходов на основе автомата A5 с 4-битным регистром глобальной истории (GHR).

Таким образом, устройство управления координирует работу всех узлов микро-ЭВМ и обеспечивает последовательное выполнение команд программы.

1.1.3 Блок регистров общего назначения

Для хранения промежуточных данных и организации вычислений предусмотрено восемь регистров общего назначения. Они используются пользователем напрямую и позволяют ускорить выполнение операций, так как доступ к регистрам значительно быстрее, чем к памяти. Управление регистрами осуществляется отдельной схемой, которая обеспечивает запись, чтение и передачу данных между регистрами и другими блоками.

1.1.4 Блок общих операций

В состав микро-ЭВМ входят базовые операции, необходимые для функционирования любой вычислительной системы:

- централизованный последовательный арбитраж к общим шинам (управляющей, адресной и данных), обеспечивающий корректное распределение доступа;
- операции NOP (пустая операция), JMP (переход), MOV adr→reg и MOV reg→adr (перемещение данных между памятью и регистрами);
- операции работы со стеком: PUSH и POP;
- операция HLT (остановка процессора).

Эти операции формируют минимальный набор команд, который гарантирует выполнение программ и управление вычислительным процессом.

1.1.5 Блок арифметико-логического устройства (АЛУ)

АЛУ выполняет все арифметические и логические операции. В его состав входят:

- арбитраж к общим шинам;
- регистры АЛУ для хранения операндов и результатов;
- базовые операции: SUB (вычитание), NOTZ (инверсия по флагу нуля), AND (логическое умножение), SRL (логический сдвиг вправо).

Особое значение имеет использование регистра флагов (FR), который хранит результаты выполнения операций (например, флаг нуля, флаг переноса). Эти данные используются устройством управления для принятия решений при условных переходах.

1.1.6 Блок стека

Стек реализован как отдельный блок памяти, организованный по принципу «последним вошёл – первым вышел» (LIFO). В данной архитектуре стек имеет направление роста вниз а его объём составляет 11 ячеек с направлением роста вниз. Стек используется для хранения адресов возврата при вызове подпрограмм, а также для временного размещения данных.

1.1.7 Блок кэша

Для ускорения работы с памятью предусмотрен кэш-блок. Его организация множественно-ассоциативная ($k=4$), что позволяет хранить несколько строк по одному адресу и уменьшает вероятность конфликтов. Алгоритм замещения строк выбран LRU (Least Recently Used), который обеспечивает эффективное использование кэша за счёт вытеснения наименее используемых данных. Синхронизация кэша с основной памятью реализована по схеме «сквозная без отображения», что упрощает проектирование, но требует аккуратного управления потоками данных.

1.1.8 Блок контроллера прямого доступа к памяти (DMA)

Контроллер DMA обеспечивает обмен данными между памятью и периферийными устройствами без участия центрального процессора. Это позволяет разгрузить процессор и повысить общую производительность системы. В состав контроллера входит схема контроля прямого доступа к памяти и механизм взаимодействия с RAM через адресное пространство. Контроллер настроен на начальный адрес 10 и объём передачи 6 байт.

1.1.9 Специальные регистры микро-ЭВМ

В архитектуре микро-ЭВМ предусмотрен ряд специальных регистров, которые обеспечивают выполнение команд и взаимодействие между блоками устройства. Эти регистры не относятся к регистрам общего назначения и не могут быть напрямую адресованы пользователем. Их содержимое формируется и используется аппаратно в процессе работы процессора, а достоверность данных в них определяется схемой выполнения операций.

IP (Instruction Pointer). Регистр указателя команд служит для хранения адреса следующей инструкции, подлежащей исполнению. Его разрядность совпадает с адресной шиной и составляет 16 бит, что позволяет адресовать до 64 Кбайт памяти. IP автоматически изменяется устройством управления при последовательном выполнении команд, а также может быть модифицирован при выполнении переходов (например, команды JMP). Таким образом, IP является ключевым элементом управления потоком выполнения программы.

IR (Instruction Register). Регистр текущей команды хранит код операции, находящейся в стадии исполнения. Разрядность регистра

определяется количеством поддерживаемых операций и для данного варианта составляет 4 бита. IR используется схемой декодирования команд, которая на основе его содержимого формирует управляющие сигналы для АЛУ, блока памяти и других узлов. Содержимое IR обновляется при каждой выборке новой инструкции из памяти.

AR (Address Register). Адресный регистр применяется в операциях, связанных с итерационной адресацией. Его разрядность равна 16 бит, что соответствует адресной шине. AR используется для хранения текущего адреса операнда или блока памяти, к которому осуществляется обращение. При выполнении операций с последовательным доступом AR может изменяться автоматически, обеспечивая итерацию по адресам.

DR (Data Register). Регистр данных служит для хранения промежуточных значений, получаемых при обмене с памятью или при выполнении операций АЛУ. Разрядность DR совпадает с шиной данных и составляет 8 бит. Он используется как накопитель: данные из памяти сначала загружаются в DR, а затем передаются в регистры общего назначения или в АЛУ для дальнейшей обработки.

FR (Flag Register). Регистр флагов хранит текущее состояние процессора и результаты выполнения операций. Его разрядность составляет 8 бит, что соответствует ширине шины данных, однако реально используется лишь часть битов (например, флаг нуля, флаг переноса, флаг отрицательного результата). FR активно применяется в арифметических и логических операциях, а также в управляющих командах, где результат выполнения зависит от состояния флагов.

Специальные регистры не могут быть напрямую адресованы пользователем и не предназначены для хранения пользовательских данных. Их использование жёстко задано схемой выполнения операций. Для хранения информации и управления вычислениями применяются регистры общего назначения и оперативная память, содержимое которых определяется пользователем и гарантируется системой.

Таким образом, специальные регистры микро-ЭВМ обеспечивают корректное выполнение команд, управление потоком программы и обработку данных, формируя основу взаимодействия между блоками устройства.

1.2 Разработка системы команд

Система команд микро-ЭВМ включает обязательный набор операций, обеспечивающих выполнение программ и взаимодействие между блоками устройства. Количество команд по заданию составляет 11.

Особенностью данной реализации является гибридная схема кодирования, при которой:

- старшие 4 бита (биты [7..4]) определяют тип операции,
- младшие 4 бита (биты [3..0]) используются для указания номера регистра общего назначения (R0–R7),

- В командах с прямой адресацией ($M \rightarrow R$, $R \rightarrow M$, JMP) после опкода следуют 2 байта адреса (старший и младший), что обеспечивает 16-битную адресацию в рамках принстонской архитектуры.

Такой подход позволяет сократить длину программы за счёт однобайтового кодирования регистровых команд и обеспечивает совместимость с 8-битной шиной данных микро-ЭВМ.

Ниже приведено описание команд, используемых в разрабатываемой микро-ЭВМ:

- NOP (00) – команда отсутствия операции; используется для синхронизации и заполнения памяти. Все неопределённые опкоды также интерпретируются как NOP, что обеспечивает отказоустойчивость системы.

- HLT (FF) – остановка процессора; выполнение программы прекращается, устройство переходит в состояние ожидания.

- JMP (20) – безусловный переход; загружает в указатель команд IP 16-битный адрес, указанный в следующих двух байтах (старший байт $\rightarrow$ младший байт). Например, 20 00 05 означает переход по адресу 0x0005.

- $M \rightarrow R$ (30–37) – загрузка данных из оперативной памяти (RAM) по прямому адресу в регистр общего назначения. Младшие 4 бита опкода определяют номер регистра: 30 $\rightarrow$ R0, 31 $\rightarrow$ R1, ..., 37 $\rightarrow$ R7. Адрес следует после опкода двумя байтами.

- $R \rightarrow M$ (40–47) – запись содержимого регистра в RAM по прямому адресу. Формат аналогичен $M \rightarrow R$.

- SUB (10–17) – вычитание: $R_n \leftarrow R_n - R_m$. Флаги: Z (ноль), S (знак), CF (перенос). Команда соответствует спецификации Варианта 8.

- SRL (50–57) – логический сдвиг вправо (Shift Right Logical). Команда соответствует спецификации Варианта 8.

- AND (60–67) – логическое И: $R_n \leftarrow R_n \text{ AND } R_m$. Команда соответствует спецификации Варианта 8.

- NOTZ (70–77) – инверсия по флагу нуля: если $Z=1$, то $R_n \leftarrow \text{NOT } R_n$. Команда соответствует спецификации Варианта 8.

- PUSH (80–87) – помещение содержимого регистра в стек (LIFO, рост вниз). Стек имеет объём 11 ячеек, базовый адрес 0x000A.

- POP (90–97) – извлечение значения из стека в регистр.

Поскольку шина данных 8-битная, каждая команда передаётся за один такт выборки. В системе используются только однобайтовые и трёхбайтовые команды:

- 1 байт: регистровые операции (NOP, HLT, SUB, SRL, AND, NOTZ, PUSH, POP),

- 3 байта: операции с прямой адресацией ($M \rightarrow R$, $R \rightarrow M$, JMP).

Порядок байтов в 16-битных адресах — big-endian (старший байт первым), что упрощает загрузку адреса в 16-битный регистр адреса за два такта без дополнительной буферизации.

Любое значение опкода, не входящее в диапазоны 00, 10–17, 20, 30–37, 40–47, 50–57, 60–67, 70–77, 80–87, 90–97, FF, распознаётся устройством управления как NOP. Это обеспечивает отказоустойчивость и возможность расширения системы команд в будущем. Диапазоны кодов операций A0–FE зарезервированы для будущих модификаций процессора. Структура системы команд микро-ЭВМ приведена в таблице 1.1.

Таблица 1.1 – Структура системы команд микро-ЭВМ

№	Команда	Длина (байт)	Формат	КОП (hex)
1	NOP	1	[00]	0
2	SUB	1	[1n]	10–17
3	JMP	3	[20][AH][AL]	20
4	M→R	3	[3n][AH][AL]	30–37
5	R→M	3	[4n][AH][AL]	40–47
6	SRL	1	[5n]	50–57
7	AND	1	[6n]	60–67
8	NOTZ	1	[7n]	70–77
9	PUSH	1	[8n]	80–87
10	POP	1	[9n]	90–97
11	HLT	1	[FF]	FF

1.3 Описание взаимодействия всех блоков микро-ЭВМ при выполнении команд программы

Главным блоком в иерархии является устройство управления (УУ). Оно выполняет выборку и декодирование команд, контролирует сохранение состояния микро-ЭВМ при ожидании завершения взаимодействия между кэшем и памятью, а также определяет, какому исполнительному блоку будет передано управление. УУ обеспечивает отсутствие коллизий при работе с общими шинами, координирует централизованный последовательный арбитраж и гарантирует согласованность работы всех узлов.

Единственным внешним сигналом, задаваемым пользователем, является тактовый сигнал CLOCK, который синхронизирует работу системы. Весь код программы хранится в ROM, и именно его содержимое полностью определяет поведение микро-ЭВМ.

После декодирования команды УУ передаёт управление соответствующему исполнительному блоку. Этот блок получает доступ к управляющей, адресной и шине данных и контролирует их до завершения всех итераций операции.

1.3.1 Исполнительные блоки:

Блок общих операций: реализует базовые команды управления потоком и памятью: NOP, JMP, M→R (загрузка из памяти в регистр), R→M (запись из

регистра в память), PUSH, POP и HLT. Управление данными командами осуществляется непосредственно сигналами устройства управления.

Блок арифметико-логического устройства (АЛУ): выполняет арифметические и логические операции над данными: SUB (вычитание), AND (логическое умножение), SRL (логический сдвиг вправо), NOT (инверсия). Для анализа результатов вычислений используется регистр флагов (FR), хранящий признаки: CF (перенос), ZF (ноль), SF (знак). Команда NOTZ реализуется путём комбинации операции NOT и проверки флага ZF в устройстве управления.

Блок стека: обеспечивает работу операций PUSH и POP. Организован по принципу «последним вошёл — первым вышел» (LIFO) в области оперативной памяти. Направление роста — вниз (к младшим адресам), объём — 11 ячеек (базовый адрес 0x000A).

Все исполнительные блоки взаимодействуют с памятью и регистрами через системные шины под контролем устройства управления, которое координирует их работу и предотвращает конфликты доступа.

1.3.2 Взаимодействие с памятью и кэшем

Обращения к памяти (ROM и RAM) осуществляются через блок кэша, который включает блок управления, блок данных и общую схему управления памятью. Кэш принимает атомарные команды чтения и записи, скрывая внутреннюю реализацию от исполнительных блоков.

Кэш-память организована как 4-канальная ассоциативная (k=4) с алгоритмом замещения LRU и сквозной записью без отображения (Write-through without allocation).

Так как в архитектуре предусмотрен контроллер DMA, возможна ситуация, когда одновременно к памяти обращается исполнительный блок через УУ и периферийное устройство через DMA. Чтобы избежать конфликтов, установлен централизованный последовательный арбитраж: исполнительные блоки в паре с УУ имеют фиксированный меньший приоритет, а периферийные устройства получают доступ к памяти напрямую. КППДП настроен на начальный адрес 10 и объём передачи 6 байт.

1.3.3 Взаимодействие с регистрами общего назначения

Блок регистров общего назначения (8 шт.) также использует общую шину данных, которую делит с блоком памяти. Арбитражом занимается исполнительный блок, захвативший шину в текущий момент. Устройство управления следит за тем, чтобы не возникали ситуации одновременного вывода данных с разных источников, предотвращая «коллизии» сигналов.

Адресация регистров ограничена диапазоном от 0 до 7. Таким образом, взаимодействие с регистрами осуществляется строго через УУ, которое задаёт порядок доступа и гарантирует корректность выполнения операций.

1.3.4 Предсказание условных переходов

Для оптимизации выполнения команд условных переходов реализована схема динамического предсказания на основе автомата А5 с 4-битным регистром глобальной истории (GHR). Таблица истории шаблонов (PHT) содержит 16 записей по 2 бита состояния, адресуемых содержимым GHR. Это позволяет повысить точность предсказания и снизить потери тактов при неправильном предсказании.

Взаимодействие всех блоков микро-ЭВМ основано на централизованном управлении через устройство управления. Исполнительные блоки получают доступ к шинам только по сигналу УУ, память обслуживается через кэш, а регистры общего назначения работают в едином арбитраже с памятью. Контроллер DMA имеет приоритетный доступ к памяти, что отражает особенности принстонской архитектуры. Такая организация обеспечивает согласованность работы системы и предотвращает конфликты при выполнении команд программы.

2 РАЗРАБОТКА ОСНОВНЫХ УСТРОЙСТВ МИКРО-ЭВМ

2.1 Запоминающие устройства

Архитектура микро-ЭВМ соответствует принстонской модели, то есть используется единая шина данных, адресная шина и шина управления для одного блока памяти. Блок памяти включает:

- модуль ROM_block (только чтение),
- модуль RAM_block (чтение/запись),
- мультиплексор lpm_mux0 для выбора источника данных.

Структурная схема блока памяти приведена в приложении А.

На схеме представлены два основных блока: ROM_block и RAM_block, подключённые к мультиплексору lpm_mux0, который по сигналу ROM_SELECT выбирает активный источник данных. Файл ROM.mif задаёт начальное содержимое ROM. Одновременно может быть активен только один блок:

- при ROM_SELECT = 1 – активен ROM,
- при ROM_SELECT = 0 – активен RAM.

Запись в RAM осуществляется по сигналу WRITE_ON:

- при WRITE_ON = 1 и фронте CLOCK данные с шины DATA_IN[7..0] записываются в адрес ADDR[15..0],

- при WRITE_ON = 0 выполняется чтение, и результат появляется на шине DATA_OUT[7..0].

Содержимое файла ROM.mif:

```
1. DEPTH = 65536;                -- Количество адресов (64K)
2. WIDTH = 8;                    -- Ширина слова (8 бит)
3. ADDRESS_RADIX = HEX;         -- Адреса в HEX
4. DATA_RADIX = HEX;           -- Данные в HEX
5.
6. CONTENT BEGIN
7.   -- 0. Инициализация (NOP)
8.   0000 : 00;                  -- NOP
9.
10.  -- 1. Загрузка данных из памяти (M->R)
11.  -- Команда 3 байта: [Опкод][Адрес Старший][Адрес Младший]
12.  0001 : 30;                  -- M->R R0, Addr=0030h
13.  0002 : 00;                  -- Адрес старший байт
14.  0003 : 30;                  -- Адрес младший байт
15.
16.  0004 : 31;                  -- M->R R1, Addr=0031h
17.  0005 : 00;
18.  0006 : 31;
19.
20.  -- 2. Арифметика и логика (1 байт)
21.  0007 : 10;                  -- SUB R0 (R0 <- R0 - R1) *
22.  0008 : 60;                  -- AND R0 (ИСПРАВЛЕНО: было 71, стало 60)
23.  0009 : 70;                  -- NOTZ R0 (ИСПРАВЛЕНО: было 80, стало 70)
24.  000A : 50;                  -- SRL R0
25.
26.  -- 3. Стек (1 байт)
27.  000B : 80;                  -- PUSH R0 (ИСПРАВЛЕНО: было A0, стало 80)
```

```

28. 000C : 92; -- POP R2 (ИСПРАВЛЕНО: было B2, стало 92)
29.
30. -- 4. Запись в память (R->M)
31. -- Команда 3 байта
32. 000D : 42; -- R->M R2, Addr=0032h
33. 000E : 00; -- Адрес старший
34. 000F : 32; -- Адрес младший
35.
36. -- 5. Переход и останов
37. 0010 : 20; -- JMP 0007h (цикл)
38. 0011 : 00; -- Адрес старший
39. 0012 : 07; -- Адрес младший
40.
41. 0013 : FF; -- HLT
42.
43. -- Заполнение остатка памяти нулями
44. [0014..FFFF] : 00;
45. END;

```

2.2 Устройство управления

Устройство управления (УУ) является центральным узлом микро-ЭВМ, обеспечивающим выборку, декодирование команд и формирование управляющих сигналов для всех исполнительных блоков. Для обеспечения читаемости и модульности схема УУ реализована иерархически с использованием символов в Quartus II, что позволяет скрыть внутреннюю сложность подблоков на схеме верхнего уровня. В состав устройства входят узел управления фазами, блок специальных регистров (IP, IR, AR) и декодер команд, которые будут подробно рассмотрены в следующих подразделах.

2.2.1 Центральный узел распределения управляющих сигналов

Центральный узел распределения управляющих сигналов предназначен для декодирования кода операции команды и формирования сигналов управления для всех функциональных блоков микро-ЭВМ: АЛУ, регистров, памяти, стека и счётчика команд.

Структурная схема блока центрального узла распределения управляющих сигналов приведена в приложении Б.

Структура узла:

- Блок дешифрации команд — 10 компараторов, каждый из которых распознаёт определённый код операции
- Блок формирования сигналов АЛУ — 3-разрядный сигнал ALU_op[2..0]
- Блок управления памятью — сигналы Mem_read, Mem_write
- Блок управления регистрами — сигнал Reg_write и адрес reg_addr[3..0]
- Блок управления стеком — сигналы push_en, pop_en
- Блок управления потоком команд — сигналы IP_inc, IP_load, BRANCH, HLT

При поступлении кода команды на вход `cmd_code[7..0]` соответствующий компаратор активируется и формирует внутренний сигнал распознавания команды. Этот сигнал поступает на логические элементы (OR, AND), которые формируют необходимые управляющие сигналы на выходах узла.

Для условной команды NOTZ дополнительно учитывается флаг нуля `flag_Z` — операция выполняется только при `flag_Z = 0`.

Декодирование команд приведено в таблице 2.1.

Таблица 2.1 – Декодирование команд

Команда	Код (bin)	Код (hex)	Активные сигналы
SUB	0001xxxx	1x	ALU_op, Reg_write
M>R	0011xxxx	3x	Mem_read, Reg_write
R>M	0100xxxx	4x	Mem_write
SRL	0101xxxx	5x	ALU_op, Reg_write
AND	0110xxxx	6x	ALU_op, Reg_write
NOTZ	0111xxxx	7x	NOTZ_en (при <code>flag_Z=0</code>)
PUSH	1000xxxx	8x	push_en
POP	1001xxxx	9x	pop_en
JMP	00100000	20	IP_load, BRANCH
HLT	11111111	FF	HLT

2.2.2 Узел местного управления фазами выполнения команд

Узел местного управления фазами (фазовый счётчик) предназначен для генерации последовательности фаз выполнения команд. Каждая команда микро-ЭВМ выполняется за несколько тактов, которые разделены на четыре фазы. Фазы выполнения команд приведены в таблице 2.2

Таблица 2.2 – Фазы выполнения команд

Фаза	Код	Назначение
FETCH	00	Выборка кода операции из памяти
ADDR_H	01	Выборка старшего байта адреса операнда
ADDR_L	10	Выборка младшего байта адреса операнда
EXEC	11	Выполнение команды

Структурная схема блока узла местного управления фазами выполнения команд приведена в приложении В.

В состав узла входят:

- 2-битный счётчик (`Phase_Counter`, `inst17`) — генерирует текущее значение фазы;
- 4 компаратора (`PHASE_FETCH_COMP`, `PHASE_ADDR_H_COMP`, `PHASE_ADDR_L_COMP`, `PHASE_EXEC_COMP`) — сравнивают значение счётчика с константами 0, 1, 2, 3;

- Логика загрузки (sload, data[1..0]) — позволяет пропускать фазы для 1-байтовых команд.

Особенности реализации. На схеме видно, что входы data[1] и data[0] подключены к источнику логической единицы (VCC) через элементы inst27 и inst28. Это обеспечивает загрузку значения "11" при активном sload, что соответствует фазе EXEC. Данное решение позволяет сократить выполнение 1-байтовых команд с 4 тактов до 2 тактов (FETCH → EXEC). Компараторы фаз приведены в таблице 2.3. Входы и выходы блока приведены в таблице 2.4

Таблица 2.3 – Компараторы фаз

Компаратор	Константа	Выходной сигнал	Активная фаза
PHASE_FETCH_COMP	0 (00)	is_fetch	Выборка команды
PHASE_ADDR_H_COMP	1 (01)	is_addr_h	Старший байт адреса
PHASE_ADDR_L_COMP	2 (10)	is_addr_l	Младший байт адреса
PHASE_EXEC_COMP	3 (11)	is_exec	Выполнение команды

Таблица 2.4 – Входы и выходы блока

Сигнал	Направление	Описание
clock	Вход	Тактовый сигнал
sload	Вход	Разрешение загрузки значения (для пропуска фаз)
data[1..0]	Вход	Значение для загрузки (подключено к "11")
phase[1..0]	Выход	Текущее значение фазы
is_fetch	Выход	Активен в фазе FETCH (00)
is_addr_h	Выход	Активен в фазе ADDR_H (01)
is_addr_l	Выход	Активен в фазе ADDR_L (10)

2.2.3 Блок указателя команд IP и блок регистра кода команд IR

Блок специальных регистров предназначен для хранения служебной информации в процессе выполнения команд. Данный блок не относится к регистрам общего назначения и не может быть напрямую адресован пользователем. Его содержимое формируется и используется аппаратно в процессе работы процессора.

Структурная схема блока указателя команд IP и блока регистра кода команд IR приведена в приложении Г.

Регистры, входящие в состав блока специальных регистров представлены в таблице 2.5.

Таблица 2.5 – Регистры блока специальных регистров

Регистр	Обозначение	Разрядность	Назначение
Указатель команд	IP (Instruction Pointer)	16 бит	Хранение адреса следующей команды, подлежащей исполнению
Регистр команды	IR (Instruction Register)	8 бит	Хранение кода операции, находящейся в стадии исполнения
Регистр старшего байта адреса	ADDR_H (Address High)	8 бит	Хранение старшего байта 16-битного адреса операнда
Регистр младшего байта адреса	ADDR_L (Address Low)	8 бит	Хранение младшего байта 16-битного адреса операнда

Разрабатываемая микро-ЭВМ соответствует принстонской модели, что означает использование единой шины данных, адресной шины и шины управления для одного блока памяти. В связи с этим, регистры IP, IR, ADDR_H и ADDR_L обеспечивают корректное взаимодействие с памятью при выборке команд и операндов.

Все регистры блока синхронизируются по общему тактовому сигналу clock. Запись данных в регистры происходит при активных сигналах загрузки (IR_load, ADDR_H_load, ADDR_L_load, IP_load). Регистр IP также поддерживает режим инкремента (IP_inc) для последовательного выполнения команд программы.

Разрядность регистра IP составляет 16 бит, что соответствует разрядности адресной шины микро-ЭВМ и позволяет адресовать до 64 Кбайт памяти ($2^{16} = 65536$ ячеек). Разрядность регистра IR составляет 8 бит, что соответствует разрядности шины данных и обеспечивает хранение кода операции и номера регистра в одном байте.

При активном сигнале глобального сброса reset = 1 все регистры блока очищаются и устанавливаются в нулевое значение. Это гарантирует известное начальное состояние устройства перед началом выполнения программы.

Блок специальных регистров взаимодействует с устройством управления, получая сигналы загрузки и инкремента от декодера команд. Регистр IP выдаёт адрес на шину адреса памяти при выборке команд. Регистр IR передаёт код операции на вход декодера команд для формирования управляющих сигналов. Регистры ADDR_H и ADDR_L формируют полный 16-битный адрес операнда для обращения к памяти в фазах ADDR_H и ADDR_L.

Особенности реализации. Для корректной работы регистра IP предусмотрена логика ИЛИ между сигналами IP_load и IP_inc, так как

компонент `lpm_counter` требует активного сигнала `clk_en` для любой операции (включая загрузку через `sload`). Данное решение обеспечивает возможность как загрузки нового адреса (для команды `JMP`), так и инкремента текущего значения (для последовательного выполнения команд).

2.2.4 Блок управления стековой памятью

Блок управления стековой памятью предназначен для организации хранения данных по принципу LIFO (Last In – First Out). Стек используется для сохранения адресов возврата при вызове подпрограмм, хранения локальных переменных и передачи параметров. Стек имеет объём 11 ячеек и растёт в сторону младших адресов (вниз). Блок обеспечивает запись данных по сигналу `push_en`, чтение по сигналу `pop_en`, а также формирование флагов переполнения и пустоты.

Структурная схема блока управления стековой памятью приведена в приложении Г.

Компоненты, входящие в состав блока управления стековой памятью представлены в таблице 2.6.

Таблица 2.6 – Компоненты блока управления стековой памятью

Компонент	Обозначение	Разрядность	Назначение
Счётчик указателя стека	SP_Counter (<code>lpm_counter</code>)	16 бит	Хранение текущего адреса вершины стека (SP)
Регистр данных	Data_Register (<code>lpm_dff</code>)	8 бит	Буферизация данных, записываемых в стек
Компаратор пустоты	CMP_Empty (<code>lpm_compare</code>)	16 бит	Сравнение SP с базовым адресом (флаг <code>empty</code>)
Компаратор полноты	CMP_Full (<code>lpm_compare</code>)	16 бит	Сравнение SP с нулевым адресом (флаг <code>full</code>)
Логика управления	OR2, NOT	1 бит	Формирование сигналов <code>clk_en</code> , <code>updown</code> , <code>sload</code>

Разрабатываемая микро-ЭВМ соответствует принстонской модели, однако стек реализован как отдельный периферийный модуль с собственным контроллером. Стек имеет объём 11 ячеек, адреса которых располагаются в диапазоне от 0 до 10. Базовый адрес стека (`stack_base`) установлен в значение 10 (0x000A), что соответствует верхней границе пустого стека при росте вниз.

Все компоненты блока синхронизируются по общему тактовому сигналу `clock`. Запись адреса в счётчик `SP_Counter` происходит синхронно по сигналу `sload`, который активируется при глобальном сбросе `reset`. Направление счёта определяется сигналом `updown`: логический 0 соответствует уменьшению адреса (PUSH), логическая 1 – увеличению (POP). Разрядность счётчика составляет 16 бит, что соответствует разрядности адресной шины микро-ЭВМ

и позволяет адресовать всё пространство памяти. Разрядность регистра данных составляет 8 бит, что соответствует разрядности шины данных.

При активном сигнале глобального сброса $reset = 1$ счётчик загружает значение `stack_base` (10), флаг `stack_empty` устанавливается в 1, флаг `stack_full` сбрасывается в 0. Это гарантирует известное начальное состояние устройства перед началом выполнения программы.

Блок управления стековой памятью взаимодействует с устройством управления процессора, получая сигналы `push_en` и `pop_en` от декодера команд. Сигнал `stack_addr` выдаётся на шину адреса памяти для обращения к ячейке стека. Сигнал `data_out` передаёт прочитанные данные на шину данных процессора.

Особенности реализации. Для корректной работы счётчика `lpm_counter` предусмотрена логика включения тактирования `clk_en`. Поскольку порт `sload` является синхронным, сигнал `clk_en` должен быть активен не только при операциях PUSH/POP, но и во время сброса для загрузки базового адреса. Поэтому сигнал `clk_en` формируется как логическое ИЛИ сигналов `reset`, `push_en` и `pop_en`. Также исключена возможность загрузки `stack_base` по сигналу `push_en` (порт `sload` подключен только к `reset`), чтобы избежать сброса указателя при каждой записи.

2.3 Арифметико-логическое устройство

2.3.1 Логическое построение АЛУ и блок РОНов

Арифметико-логическое устройство (АЛУ) является основным исполнительным узлом микро-ЭВМ, предназначенным для выполнения арифметических и логических операций над данными. АЛУ реализует следующие операции: вычитание (SUB), логическое И (AND), логический сдвиг вправо (SRL) и условную инверсию (NOTZ). Блок регистров общего назначения (РОН) предназначен для хранения 8-разрядных операндов и результатов выполнения команд, блок РОН содержит 8 регистров разрядностью 8 бит каждый.

В разработанной микро-ЭВМ используется архитектура с неявным аккумулятором. В качестве аккумулятора используется регистр R0, который имеет прямой вывод на шину данных. Это обеспечивает постоянную доступность аккумулятора для двухоперандных команд (SUB, AND) и позволяет выполнять чтение обоих операндов за один такт.

Структурная схема блока РОНов приведена в приложении Е.

Компоненты, входящие в состав блока РОНов представлены в таблице 2.7

Таблица 2.7 – Компоненты блока РОН

Компонент	Обозначение	Разряд-ность	Назначение
Регистр 8-битный	reg_8bit (lpm_dff)	8 бит	Хранение одного операнда
Дешифратор адреса	lpm_decode_3to8	3→8	Выбор регистра для записи
Мультиплексор операнда	mux_operand_8to1 (lpm_mux)	8 бит	Выбор второго операнда по адресу
Элемент И	AND2	1 бит	Формирование сигнала разрешения записи (enable)
Регистр 8-битный	reg_8bit (lpm_dff)	8 бит	Хранение одного операнда

Принцип работы РОН: запись данных осуществляется только в тот регистр, адрес которого совпадает с сигналом Addr_W[2..0] при активном сигнале Reg_Write. Выбор регистра для записи производится дешифратором 3→8. Чтение данных осуществляется через два канала: первый канал (ACC_Out) подключён напрямую к выходу регистра R0 (аккумулятор), второй канал (Operand) реализован на основе мультиплексора 8→1, который выбирает данные из любого регистра R0...R7 на основе адреса Addr_Op[2..0].

Блок регистра 8 бит (reg_8bit) предназначен для хранения 8-разрядного слова данных. Каждый бит регистра реализован на основе D-триггера с синхронным тактированием и асинхронным сбросом. Регистр имеет вход разрешения записи (enable), который позволяет записывать данные только при активном сигнале разрешения. Выходное слово формируется как $q[i] = data[i]$ для $i = 0..7$ при условии $enable = 1$ и положительном фронте тактового сигнала.

Дешифратор адреса (lpm_decode_3to8) выполняет преобразование 3-разрядного двоичного кода адреса в один из восьми активных выходов. При подаче на вход data[2..0] кода от 000 до 111 активируется соответствующий выход eq[0]...eq[7]. Дешифратор реализован на основе стандартного компонента библиотеки LPM. Выходы дешифратора подключаются к элементам И для формирования индивидуальных сигналов разрешения записи каждого регистра.

Мультиплексор операнда (mux_operand_8to1) осуществляет выбор одного из восьми 8-разрядных входов на основе 3-разрядного адреса Addr_Op[2..0]. Компонент реализован на основе параметризованного модуля LPM_MUX с настройками: 8 входов данных, ширина данных 8 бит. Выходное слово формируется по правилу $result[7..0] = dataN[7..0]$, где N — значение сигнала выбора sel[2..0].

Особенности реализации. В разработанном блоке РОН реализована архитектура со встроенным аккумулятором. Регистр R0 выполняет функцию аккумулятора и имеет прямой вывод на шину ACC_Out[7..0] без использования мультиплексора. Это обеспечивает постоянную доступность

аккумулятора для двухоперандных команд (SUB, AND) и позволяет выполнять чтение обоих операндов за один такт. Сигнал разрешения записи для каждого регистра формируется как логическое И между выходом дешифратора и общим сигналом Reg_Write: $\text{enable_Ri} = (\text{Addr_W} == i) \text{ AND Reg_Write}$. Асинхронный сброс reset подключён ко всем регистрам одновременно и устанавливает их содержимое в ноль при активном уровне сигнала.

2.3.2 Реализация арифметических и логических операций

Блок арифметико-логического устройства (АЛУ) предназначен для выполнения арифметических и логических операций над 8-разрядными операндами. Кодирование операций АЛУ приведено в таблице 2.8.

Таблица 2.8 – Кодирование операций АЛУ

Код операции	Мнемоника	Описание
000	SUB	Вычитание: $\text{result} = \text{data_a} - \text{data_b}$
001	AND	Логическое И: $\text{result} = \text{data_a} \text{ AND } \text{data_b}$
010	SRL	Логический сдвиг вправо: $\text{result} = \text{data_a} \gg 1$
011	NOT	Инверсия: $\text{result} = \text{NOT data_a}$

Структурная схема блока АЛУ приведена в приложении Ж.

Компоненты блока арифметических и логических операций представлены в таблице 2.9

Таблица 2.9 – Компоненты блока арифметических и логических операций

Компонент	Обозначение	Разрядность	Назначение
Блок вычитания	alu_subtractor (lpm_add_sub)	8 бит	Выполнение операции вычитания $\text{data_a} - \text{data_b}$
Блок сдвига вправо	alu_shifter_sr (lpm_shiftreg)	8 бит	Логический сдвиг операнда вправо на 1 позицию
Мультиплексор результата	alu_output_mux (lpm_mux)	8 бит	Выбор результата операции по коду $\text{alu_op}[2..0]$
Блок инверсии*	alu_logic_not8	8 бит	Поразрядная инверсия операнда data_a
Блок логического И*	alu_logic_and8	8 бит	Поразрядное логическое умножение data_a и data_b
Блок формирования флага Z*	alu_logic_flag_zero	1 бит	Генерация сигнала flag_Z при $\text{result} = 0$

* – компоненты, разработанные самостоятельно

Принцип работы АЛУ: в зависимости от кода alu_op один из четырёх функциональных блоков формирует результат, который через выходной мультиплексор передаётся на шину result. Параллельно формируется набор флагов, отражающих свойства результата.

Компоненты alu_logic_not8, alu_logic_and8 и alu_logic_flag_zero не входят в стандартную библиотеку LPM и были разработаны самостоятельно в среде Quartus II. Ниже приведено их описание и структурные схемы.

Блок инверсии (alu_logic_not8) предназначен для инверсии 8-разрядного входного слова. Каждый бит входа data[7..0] инвертируется независимо. Выходное слово формируется как $result[i] = NOT\ data[i]$ для $i = 0..7$. Реализация выполнена на основе 8 параллельных вентилей NOT, подключённых к соответствующим разрядам входной шины. Структурная схема блока alu_logic_not8 приведена на рисунке 2.1.

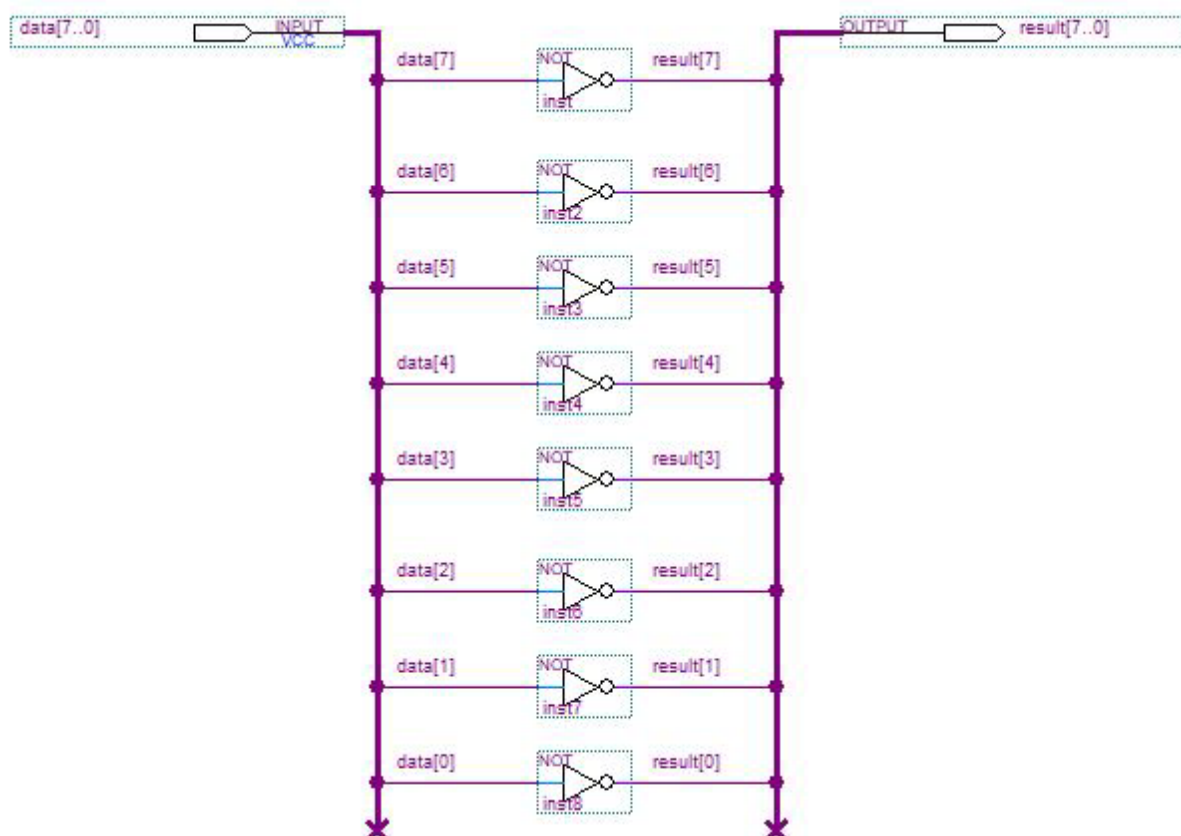


Рисунок 2.1 – Структурная схема блока alu_logic_not8

Блок логического умножения (alu_logic_and8) выполняет поразрядное логическое И двух 8-разрядных операндов. Выходное слово формируется по правилу $result[i] = dataa[i] AND\ datab[i]$ для каждого разряда шины данных. Схема реализована с использованием 8 параллельных вентилей AND, обеспечивающих независимую обработку битов операндов dataa и datab. Структурная схема блока alu_logic_and8 приведена на рисунке 2.2.

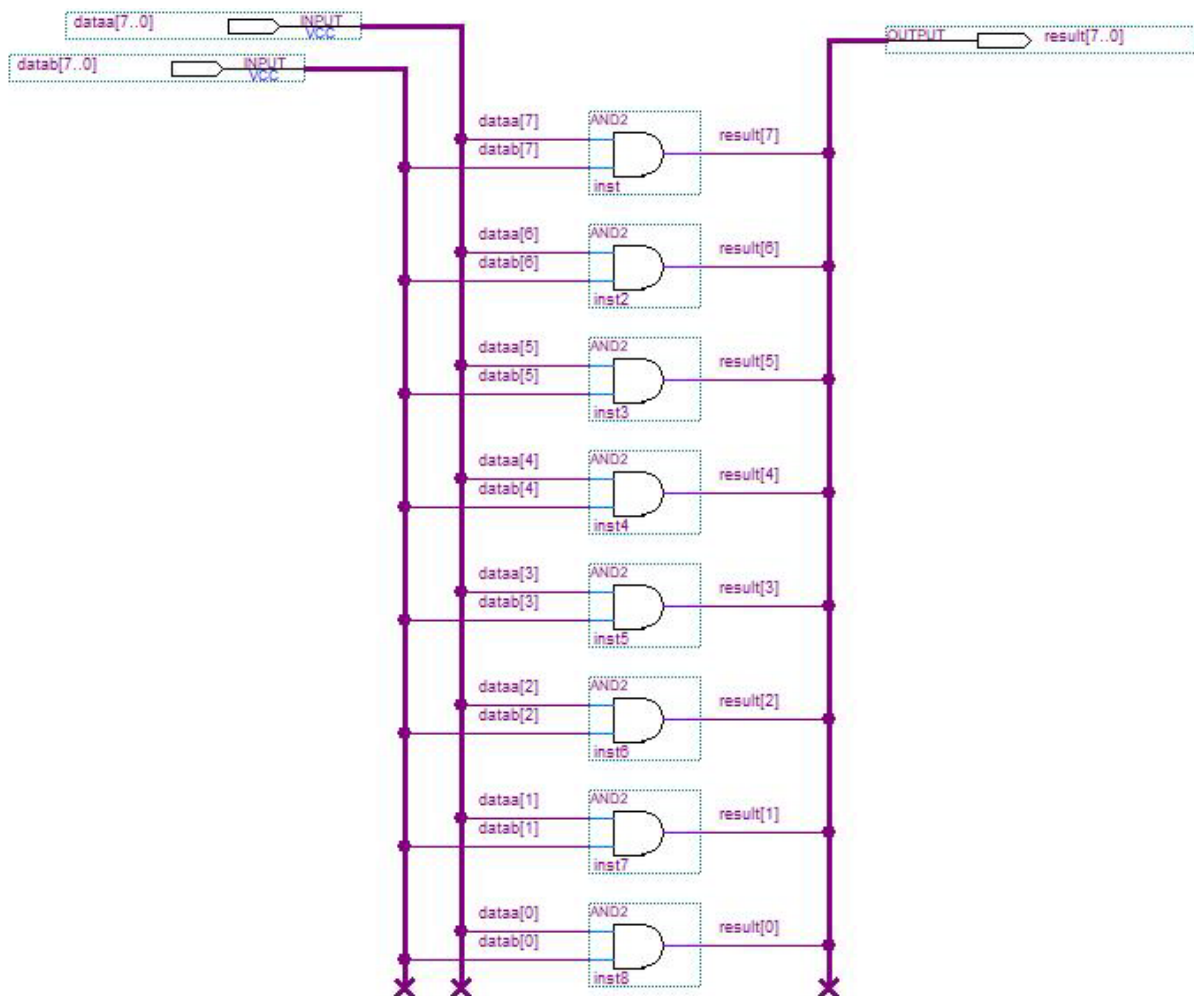


Рисунок 2.2 – Структурная схема блока alu_logic_and8

Блок формирования флага нуля (alu_logic_flag_zero) генерирует сигнал $\text{flag_Z} = 1$ в случае, когда результат операции равен нулю. Принцип работы основан на проверке всех битов входного слова $\text{data}[7..0]$ на равенство нулю с помощью вентиля NOR. Если все биты входа равны 0, на выходе формируется логическая единица, в противном случае – ноль. Структурная схема блока alu_logic_flag_zero приведена на рисунке 2.3.

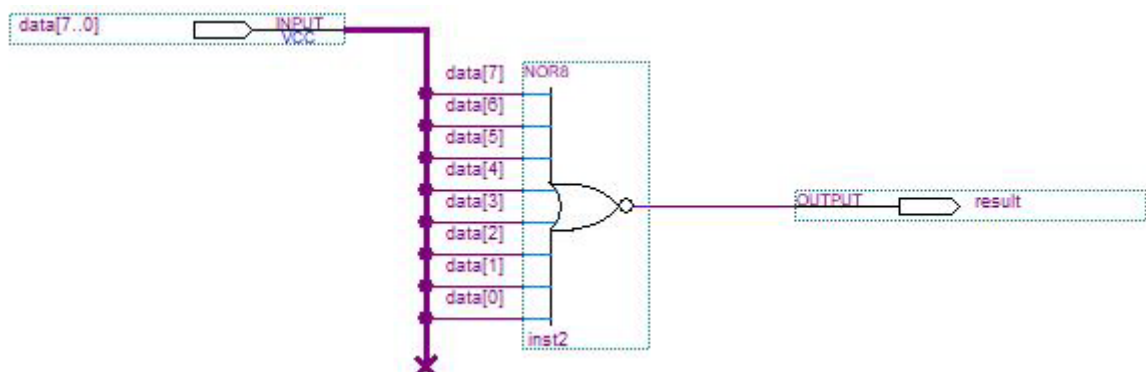


Рисунок 2.3 – Структурная схема блока alu_logic_flag_zero

Особенности реализации. Для корректного формирования флага переноса flag_CF предусмотрена инверсия сигнала cout от блока вычитания alu_subtractor, поскольку в компоненте lpm_add_sub сигнал cout = 1 генерируется при отсутствии займа (когда data_a $\geq$ data_b). Для активации флага CF только при операции вычитания предусмотрена логика AND между инвертированным cout и сигналом, формируемым при alu_or = "000". Флаг нуля flag_Z формируется на основе вентиля NOR от всех битов результата: если result[7..0] = 0, то flag_Z = 1. Флаг знака flag_S подключён напрямую к старшему биту результата result[7].

2.4 Организация кэш-памяти процессора

Кэш-память является быстродействующим буфером между центральным процессором и основной памятью, предназначенным для хранения наиболее часто используемых данных и команд. Применение кэш-памяти позволяет существенно повысить производительность микро-ЭВМ за счёт сокращения среднего времени доступа к данным.

В разработанной микро-ЭВМ используется 4-канальная множественно-ассоциативная кэш-память (4-way set associative cache) с организацией Write-Through (сквозная запись). Каждый блок данных из основной памяти может быть размещён в любой из четырёх строк (ways) внутри набора (set), что обеспечивает компромисс между скоростью работы и эффективностью использования памяти.

Структурная схема блока кэш-памяти приведена в приложении И.

Компоненты, входящие в состав блока кэш-памяти, представлены в таблице 2.10.

Таблица 2.10 – Компоненты блока кэш-памяти процессора

Компонент	Обозначение	Разрядность	Назначение
RAM данных (Way 0–3)	cache_data_way X (lpm_ram_dp)	8 бит $\times$ 4 слова	Хранение данных для 4 каналов
RAM тэгов (Way 0–3)	cache_tag_way X (lpm_ram_dp)	10 бит $\times$ 4 слова	Хранение тэгов для 4 каналов
RAM бит валидности (Way 0–3)	cache_valid_wayX (lpm_ram_dp)	1 бит $\times$ 4 слова	Флаги валидности строк кэша
Компаратор тэгов (Way 0–3)	cache_tag_compare_wayX	10 бит	Сравнение тэга адреса с тэгом канала
Мультиплексор данных	cache_data_mux (lpm_mux)	4 $\rightarrow$ 1, 8 бит	Выбор данных из канала с попаданием

Счётчик LRU	lru_counter (up counter)	2 бита	Отслеживание порядка использования наборов
Дешифратор LRU	lru_decoder	2→4	Преобразование счётчика в сигналы выбора канала (eq0–eq3)
Элемент И 3 входа	AND3 (lpm_and)	3 входа	Формирование сигналов записи wren_tagX
Элемент И 2 входа	AND2 (lpm_and)	2 входа	Логика управления записью
Элемент ИЛИ 2 входа	OR2 (lpm_or)	2 входа	Формирование сигналов cache_hit/cache_miss
Элемент НЕ	NOT (lpm_not)	1 вход	Инверсия сигналов управления

Блок RAM данных (cache_data_wayX) предназначен для хранения 8-разрядных слов данных. Каждый из четырёх каналов содержит RAM объёмом 16 слов по 8 бит. Блок имеет входы адреса записи wraddress[3..0] и адреса чтения rdaddress[3..0], которые формируются из битов адреса процессора cru_addr[5..2]. Запись данных осуществляется при активном сигнале разрешения записи wren на положительном фронте тактового сигнала clock. Выходные данные q[7..0] формируются из ячейки памяти, адрес которой задан на входе rdaddress.

Блок RAM тэгов (cache_tag_wayX) хранит 10-разрядные тэги для идентификации блоков основной памяти. Каждый канал содержит RAM объёмом 16 слов по 10 бит. Тэг формируется из старших битов адреса процессора cru_addr[15..6] и записывается в кэш при промахе (Cache Miss). При чтении тэг из кэша подаётся на компаратор для сравнения с текущим адресом процессора. Выходные данные tag_q[9..0] представляют собой тэг хранящейся в данной строке кэша.

Блок RAM бит валидности (cache_valid_wayX) хранит флаги валидности строк кэш-памяти. Каждый канал содержит RAM объёмом 16 слов по 1 бит. Бит валидности устанавливается в 1 при первой загрузке данных в строку кэша и сбрасывается в 0 при reset. Проверка бита валидности осуществляется совместно с проверкой тэга — попадание фиксируется только при одновременном совпадении тэга и установленном бите валидности. Выходной сигнал valid[0] формируется из ячейки памяти, адрес которой задан на входе rdaddress.

Компаратор тэгов (cache_tag_compare_wayX) осуществляет сравнение 10-разрядного тэга из адреса процессора с тэгом, хранящимся в соответствующем канале кэша. Компонент реализован на основе поразрядного сравнения с формированием сигнала равенства aeb (all equal bits). При полном совпадении всех 10 бит компаратор выдаёт логическую единицу на выход cmp_tagX. Сигнал cmp_tagX поступает на элементы И для формирования окончательного сигнала попадания cache_hit.

Мультиплексор данных (cache_data_mux) осуществляет выбор одного из четырёх 8-разрядных входов на основе 2-разрядного адреса sel[1..0]. Компонент реализован на основе стандартного компонента библиотеки LPM. Входы data0x...data3x подключены к выходам RAM данных всех четырёх каналов. Адрес выбора sel[1..0] формируется дешифратором LRU и указывает на канал, в котором произошло попадание. Выходное слово result[7..0] передаётся на выход cpu_data_out[7..0].

Счётчик LRU (lru_counter) отслеживает порядок использования каналов кэш-памяти для выбора канала для замещения при промахе. Компонент реализован на основе 2-разрядного реверсивного счётчика с синхронным тактированием и асинхронным сбросом. Счётчик инкрементируется при каждом обращении к кэшу и хранит значение от 0 до 3, соответствующее наименее недавно использованному каналу. Выходное значение q[1..0] подаётся на дешифратор LRU.

Дешифратор LRU (lru_decoder) преобразует 2-разрядное значение счётчика LRU в один из четырёх активных выходов eq0...eq3. При подаче на вход data[1..0] кода от 00 до 11 активируется соответствующий выход eqX. Выходы дешифратора подключаются к элементам И для формирования индивидуальных сигналов разрешения записи wren_tagX каждого канала. Сигнал eqX указывает на канал, который будет замещён при следующем промахе кэша.

Логика управления записью формирует сигналы разрешения записи wren_tagX и data_wrenX на основе сигналов mem_read, mem_write, cache_hit, cache_miss и cmp_tagX. Сигнал wren_tagX активируется при промахе чтения (mem_read AND cache_miss AND eqX) для записи нового тэга в выбранный LRU канал. Сигнал data_wrenX активируется при промахе чтения для загрузки данных из основной памяти или при попадании записи (mem_write AND cache_hit AND cmp_tagX) для обновления данных в кэше. Логика реализована на основе элементов И 2-входовых (AND2) и 3-входовых (AND3).

Сигналы попадания cache_hit и промаха cache_miss формируются на основе результатов сравнения тэгов и проверки бит валидности. Сигнал cache_hit активируется при одновременном выполнении условий cmp_tagX = 1 и validX = 1 хотя бы для одного из четырёх каналов. Логика реализована на основе элементов ИЛИ 2-входовых (OR2) с последующей инверсией для формирования cache_miss. Сигнал cache_miss используется для инициирования обращения к основной памяти и загрузки данных в кэш.

2.5 Описание системы предсказания переходов

Блок динамического предсказания условных переходов предназначен для повышения производительности конвейера команд центрального процессора за счёт заблаговременного прогнозирования исхода команд условного перехода. Применение данного блока позволяет минимизировать

простой конвейера, связанные с необходимостью ожидания вычисления условия перехода.

В разработанной микро-ЭВМ используется двухбитный адаптивный предсказатель на основе автомата А5 с использованием регистра глобальной истории (GHR – Global History Register). Данный подход обеспечивает высокую точность предсказания за счёт учёта истории последних четырёх команд условного перехода независимо от их адреса в программе.

Структурная схема блока предсказания переходов приведена в приложении К.

Компоненты, входящие в состав блока предсказания переходов, представлены в таблице 2.11.

Таблица 2.11 – Компоненты блока предсказания переходов

Компонент	Обозначение	Разрядность	Назначение
Регистр глобальной истории	ghr_register (lpm_shiftreg)	4 бита	Хранение истории последних 4-х переходов
Таблица истории шаблонов	pht_table (lpm_ram_dq)	2 бита × 16 слов	Хранение состояний автомата А5 для каждого шаблона
Комбинационная схема логики	AND2, OR2, NOT	–	Формирование следующего состояния автомата

Блок регистра глобальной истории (ghr_register) реализован на основе сдвигового регистра и предназначен для хранения информации об исходах последних четырёх команд условного перехода. При каждом подтверждённом переходе (сигнал branch_valid = 1) в младший разряд регистра записывается фактический исход команды (сигнал branch_taken: 1 – переход состоялся, 0 – переход не состоялся), а предыдущие биты сдвигаются влево. Выходной сигнал ghr_out[3..0] формирует адрес для обращения к таблице истории шаблонов.

Блок таблицы истории шаблонов (pht_table) представляет собой однопортовую RAM память объёмом 16 слов по 2 бита. Каждая ячейка памяти хранит состояние двухбитного автомата А5 для соответствующего шаблона глобальной истории. Адресом для чтения и записи служит текущее значение регистра GHR. При поступлении новой команды условного перехода из ячейки РНТ считывается текущее состояние автомата, старший бит которого (q[1]) формирует сигнал предсказания prediction: если q[1] = 1, предсказывается переход, если q[1] = 0 – переход не предсказывается.

Комбинационная схема логики автомата А5 реализует функцию формирования следующего состояния на основе текущего состояния и фактического исхода перехода. Логика работы описывается следующими уравнениями:

- $d1 = \text{branch_taken} \text{ OR } (q1 \text{ AND } q0)$
- $d0 = (q1 \text{ AND NOT } q0) \text{ OR } (\text{branch_taken} \text{ AND } q0)$

где $q1$, $q0$ – биты текущего состояния автомата; $d1$, $d0$ – биты следующего состояния; branch_taken – фактический исход перехода.

Запись нового состояния в таблицу PHT производится при активном сигнале branch_valid на положительном фронте тактового сигнала clock . Сигнал $\text{new_state}[1..0]$, сформированный комбинационной схемой, записывается в ячейку памяти по адресу, заданному текущим значением $\text{ghr_out}[3..0]$.

Автомат A5 может находиться в четырёх состояниях:

- 00 – сильное предсказание «перехода не будет»;
- 01 – слабое предсказание «перехода не будет»;
- 10 – слабое предсказание «переход будет»;
- 11 – сильное предсказание «переход будет».

Переходы между состояниями осуществляются в соответствии с фактическим исходом команды условного перехода, что обеспечивает адаптацию предсказателя к поведению конкретной программы.

2.6 Аппаратура и функционирование КПДП

Блок контроллера прямого доступа к памяти (КПДП, DMA-контроллер) предназначен для организации передачи данных между периферийными устройствами и памятью без участия центрального процессора. Применение данного блока позволяет разгрузить процессор от выполнения операций ввода-вывода, повышая общую производительность системы за счёт параллельного выполнения вычислений и передачи данных.

В разработанной микро-ЭВМ используется КПДП с автоматическим управлением передачей на основе конечного автомата (FSM – Finite State Machine). Данный подход обеспечивает автономную работу контроллера после получения команды запуска, минимизируя взаимодействие с процессором в процессе передачи данных.

Структурная схема блока КПДП приведена в приложении Л.

Компоненты, входящие в состав блока КПДП, представлены в таблице 2.12.

Таблица 2.12 – Компоненты блока КПДП

Компонент	Обозначение	Разрядность	Назначение
Конечный автомат управления	fsm_rom (lpm_rom)	5 бит × 16 слов	Хранение таблицы переходов и управляющих сигналов FSM

Регистр состояния FSM	state_register (DFF)	2 бита	Хранение текущего состояния автомата
Регистр адреса передачи	addr_register (DFF)	8 битов	Хранение текущего адреса памяти для передачи
Счётчик оставшихся байтов	count_register (DFF)	3 бита	Подсчёт количества переданных байтов
Сумматор адреса	addr_adder (ADD)	8 битов	Инкремент адреса при каждой передаче
Вычитатель счётчика	count_subtractor (SUB)	3 бита	Декремент счётчика после каждой передачи
Компаратор нуля	count_zero_cmp (CMP)	3 бита	Формирование флага окончания передачи
Мультиплексор адреса	addr_mux (MUX)	8 битов	Выбор между загрузкой начального адреса и инкрементом
Мультиплексор счётчика	count_mux (MUX)	3 бита	Выбор между загрузкой начального значения и декрементом
Логика управления	AND2, OR2, NOT	–	Формирование управляющих сигналов

Блок конечного автомата (fsm_rom) реализован на основе ПЗУ и содержит таблицу переходов между четырьмя состояниями: IDLE (00) – ожидание команды запуска, REQUEST (01) – запрос захвата шины, TRANSFER (10) – активная передача данных, COMPLETE (11) – завершение передачи. Адресация в ROM формируется из текущего состояния (2 бита), сигнала запуска dmac_start (1 бит) и флага окончания счётчика count_is_zero (1 бит), что обеспечивает 16 возможных комбинаций адресов. Выходные данные FSM (5 бит) включают: следующее состояние (2 бита), сигнал активности передачи transfer_active (1 бит), сигнал загрузки счётчика count_load_sel (1 бит) и сигнал загрузки адреса addr_load_sel (1 бит). Содержимое файла ROM.mif:

```

1. WIDTH = 5;
2. DEPTH = 16;
3. ADDRESS_RADIX = DEC;
4. DATA_RADIX = BIN;
5.
6. CONTENT BEGIN
7.
8.     -- СОСТОЯНИЕ 00 (IDLE) - адреса 0-3
9.     0 : 00000; -- start=0, zero=0: IDLE→IDLE
10.    1 : 01100; -- start=0, zero=1: IDLE→IDLE
11.    2 : 00101; -- start=1, zero=0: IDLE→REQUEST
12.    3 : 00101; -- start=1, zero=1: IDLE→REQUEST
13.

```

```

14.    -- СОСТОЯНИЕ 01 (REQUEST) - адреса 4-7
15.    4 : 00010; -- REQUEST→TRANSFER
16.    5 : 00010;
17.    6 : 00010;
18.    7 : 00010;
19.
20.    -- СОСТОЯНИЕ 10 (TRANSFER) - адреса 8-11
21.    8 : 10010; -- [1][0][0][1][0] = trans=1, cnt_ld=0, addr_ld=0,
    next=10
22.    9 : 00111; -- zero=1: TRANSFER→COMPLETE
23.    10: 10010; -- zero=0: TRANSFER→TRANSFER
24.    11: 00111; -- zero=1: TRANSFER→COMPLETE
25.
26.    -- СОСТОЯНИЕ 11 (COMPLETE) - адреса 12-15
27.    12: 00100; -- COMPLETE→IDLE
28.    13: 00100;
29.    14: 00100;
30.    15: 00100;
31.
32. END;

```

Формат выходных данных FSM:

- [4] – transfer_active (сигнал активной передачи);
- [3] – count_load_sel (загрузка счётчика);
- [2] – addr_load_sel (загрузка адреса);
- [1] – next_state[1] (старший бит следующего состояния);
- [0] – next_state[0] (младший бит следующего состояния).

Работа блока КПДП начинается с состояния IDLE, в котором контроллер ожидает поступления сигнала запуска dmac_start. В данном состоянии регистр адреса и счётчик байтов удерживают начальные значения, сформированные через мультиплексоры загрузки. Адресный регистр хранит начальное значение 0x0A, а счётчик хранит значение 6, что соответствует передаче семи байтов данных. При поступлении сигнала dmac_start = 1 конечный автомат переходит в состояние REQUEST, в котором формируется сигнал request для запроса захвата системной шины.

После получения разрешения на работу с шиной контроллер переходит в состояние TRANSFER, в котором происходит непосредственная передача данных. В этом состоянии активируется сигнал transfer_active, который разрешает работу счётчика и адресного регистра. Сигнал mem_read формируется для чтения данных из памяти по текущему адресу, хранящемуся в адресном регистре. После каждой успешной передачи адрес инкрементируется на единицу для перехода к следующей ячейке памяти, а счётчик оставшихся байтов декрементируется. Сигнал dmac_busy находится в активном состоянии на протяжении всей передачи, информируя систему о занятости контроллера.

Когда счётчик оставшихся байтов достигает нулевого значения, компаратор нуля формирует сигнал count_is_zero = 1, который поступает на вход адресации FSM ROM. Конечный автомат переходит в состояние COMPLETE, в котором формируется сигнал завершения передачи dmac_done.

В этом состоянии адресный регистр сбрасывается в начальное значение через мультиплексор загрузки адреса, подготавливая контроллер к следующей операции передачи.

После завершения состояния COMPLETE контроллер автоматически возвращается в состояние IDLE, в котором ожидает следующего сигнала запуска. Сигнал `dmac_busy` сбрасывается, освобождая системную шину для других устройств. Такой цикл работы обеспечивает автономную передачу блоков данных без постоянного контроля со стороны центрального процессора, что позволяет процессору выполнять другие задачи в процессе работы DMA-контроллера.

2.7 Описание арбитра шины

Блок арбитра шины предназначен для координации доступа нескольких masters (центрального процессора и контроллера прямого доступа к памяти) к общей системной шине данных и адреса. Применение данного блока позволяет избежать конфликтов при одновременных запросах на доступ к шине, обеспечивая упорядоченную передачу данных между устройствами.

В разработанной микро-ЭВМ используется централизованный последовательный арбитраж с фиксированным приоритетом. Данный подход обеспечивает простоту реализации и детерминированное время доступа для процессора, который имеет высший приоритет по сравнению с другими мастерами шины.

Структурная схема блока арбитража шины приведена в приложении М.

Компоненты, входящие в состав блока арбитража шины, представлены в таблице 2.13.

Таблица 2.13 – Компоненты блока КПДП

Компонент	Обозначение	Разрядность	Назначение
Регистр запроса CPU	<code>req_cpu_reg (LPM_DFF)</code>	1 бит	Синхронизация сигнала <code>request_cpu</code> с системным тактовым сигналом
Регистр запроса DMA	<code>req_dmac_reg (LPM_DFF)</code>	1 бит	Синхронизация сигнала <code>request_dmac</code> с системным тактовым сигналом
Логика формирования разрешений	<code>priority_logic</code>	3 бита	Формирование сигналов <code>grant_cpu</code> , <code>grant_dmac</code> и <code>bus_busy</code> на основе синхронизированных запросов

Блок синхронизации запросов реализован на основе двух D-триггеров (`LPM_DFF`) с синхронным сбросом. При каждом положительном фронте тактового сигнала значения сигналов `request_cpu` и `request_dmac` записываются в соответствующие регистры, что обеспечивает их синхронизацию с системной частотой и исключает возникновение метастабильных состояний.

Логика формирования разрешений реализует приоритет процессора над DMA-контроллером. Сигнал `grant_cpu` формируется путём прямой передачи синхронизированного запроса от процессора (`req_cpu_sync`). Сигнал `grant_dmac` формируется только при одновременном выполнении двух условий: наличии запроса от DMA (`req_dmac_sync = 1`) и отсутствии запроса от процессора (`req_cpu_sync = 0`). Это обеспечивается схемой: `grant_dmac = req_dmac_sync AND NOT req_cpu_sync`.

Сигнал занятости шины `bus_busy` формируется как логическая сумма сигналов `grant_cpu` и `grant_dmac`: `bus_busy = grant_cpu OR grant_dmac`. Данный сигнал находится в активном состоянии (логическая единица) на протяжении всего времени, пока один из мастеров удерживает доступ к шине, и блокирует возможность предоставления доступа другим мастерам.

Работа блока арбитража происходит следующим образом. При поступлении запроса от одного из мастеров (`request_cpu = 1` или `request_dmac = 1`) соответствующий сигнал синхронизируется в регистре на следующем положительном фронте тактового сигнала. Если шина свободна (`bus_busy = 0`), арбитр формирует разрешение (`grant`) для запрашивающего мастера. При одновременном поступлении запросов от CPU и DMA приоритет отдаётся процессору — формируется `grant_cpu = 1`, а запрос от DMA остаётся необслуженным до тех пор, пока процессор не завершит свою операцию и не снимет сигнал `request_cpu`.

После завершения операции мастер снимает сигнал запроса (`request = 0`), что приводит к сбросу соответствующего сигнала разрешения (`grant = 0`) и сигнала занятости шины (`bus_busy = 0`), если другой мастер не удерживает доступ. Арбитр готов к обслуживанию следующего запроса.

3 ФУНКЦИОНАЛЬНОЕ МОДЕЛИРОВАНИЕ

3.1 Функциональное моделирование блока ОЗУ

Результаты функционального моделирования блока ОЗУ показаны на рисунке 3.1.

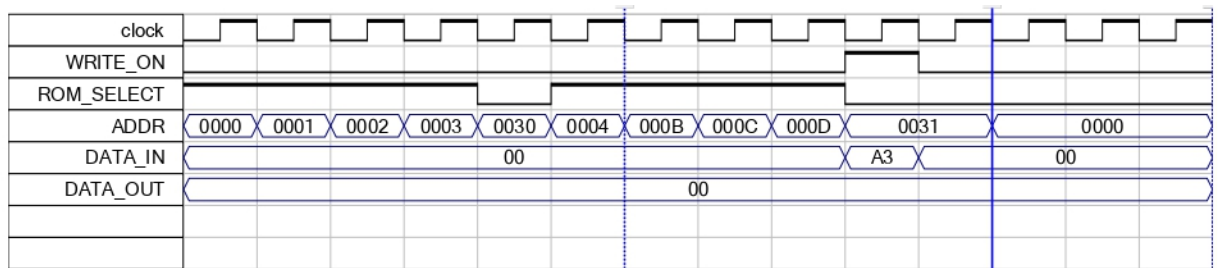


Рисунок 3.1 – Результаты функционального моделирования

Пошаговое описание по тактам (T):

Команда 1 – M→R [0x0030] (ROM: 30 00 30):

T0 (0–10 ns). На входную шину адреса устанавливаем ADDR = 0001, включаем чтение из ROM (ROM_SELECT = 1) и запрещаем запись (WRITE_ON = 0); DATA_IN находится в три стейте. Это означает: процессор запрашивает первый байт инструкции – opcode. В течение этого такта из ROM на шину данных появится байт 0x30 (с учётом задержки модели ≈ 2 ns), который фиксируется как opcode для последующего декодирования.

T1 (10–20 ns). Инкрементируем адрес на следующий байт инструкции: ADDR = 0002, остаёмся в режиме чтения ROM (ROM_SELECT = 1), запись по прежнему запрещена (WRITE_ON = 0). В этот такт считываем старший байт операнда – 0x00. Этот байт поступает на DATA_OUT примерно через 2 ns после установки адреса и сохраняется как старшая часть 16 битного адреса операнда.

T2 (20–30 ns). Снова инкремент адреса: ADDR = 0003, ROM_SELECT = 1, WRITE_ON = 0. Читаем младший байт операнда – 0x30. После получения этого байта декодер собирает полный 16 битный адрес операнда: ADDR_OPERAND = $0x00 \ll 8 \mid 0x30 = 0x0030$. На этом этапе завершён fetch цикл инструкции (opcode + операнды) и начинается стадия decode.

T3 (30–40 ns). Для выполнения операции переключаем источник данных на RAM: ADDR = 0030, ROM_SELECT = 0, WRITE_ON = 0, DATA_IN = ZZ. Это означает: запрос содержимого RAM по адресу 0x0030. Через небольшую задержку (≈ 2 ns) на DATA_OUT появится значение RAM[0x0030]; это значение фиксируем на волновой форме как результат выполнения команды M→R.

Промежуток подготовки (40–60 ns). ADDR = 0004, ROM_SELECT = 1, WRITE_ON = 0 – подготовка к следующей инструкции и стабилизация сигналов.

Команда 2 – R→M [0x0031] (ROM: 42 00 31)

T0 (60–70 ns). На входную шину адреса устанавливаем ADDR = 000B, включаем чтение из ROM (ROM_SELECT = 1) и запрещаем запись (WRITE_ON = 0); DATA_IN находится в три стейте (ZZ). Процессор запрашивает первый байт инструкции – opcode. В течение этого такта из ROM на шину данных появится байт 0x42 с модельной задержкой ≈ 2 ns; этот байт фиксируется в IR как opcode для последующего декодирования.

T1 (70–80 ns). Инкрементируем адрес на следующий байт инструкции: ADDR = 000C, остаёмся в режиме чтения ROM (ROM_SELECT = 1), запись по прежнему запрещена (WRITE_ON = 0). В этот такт считываем старший байт 16 битного адреса операнда – 0x00. Значение поступает на DATA_OUT примерно через 2 ns после установки адреса и сохраняется в буфере операндов как ADDR_HIGH.

T2 (80–90 ns). Снова инкремент адреса: ADDR = 000D, ROM_SELECT = 1, WRITE_ON = 0. Читаем младший байт адреса – 0x31. После появления этого байта декодер собирает полный 16 битный адрес операнда: ADDR_OPERAND = $(0x00 \ll 8) \mid 0x31 = 0x0031$. На этом этапе завершён fetch цикл инструкции (opcode + операнды) и формируются микрокоманды для execute (запись в RAM).

T3 (90–100 ns). Для выполнения операции переключаем источник данных на RAM: ADDR = 0031, ROM_SELECT = 0. Готовим запись: DATA_IN = 0xA3, WRITE_ON = 1. В течение этого такта мультиплексор адреса подключает RAM, подаётся стабильный DATA_IN = 0xA3 и активируется WRITE_ON на один такт; RAM записывает байт 0xA3 по адресу 0x0031. Важно, чтобы DATA_IN был стабилен до и в течение интервала 90–100 ns, а ROM_SELECT был равен 0 во время записи.

T4 (100–110 ns). Снимаем WRITE_ON (WRITE_ON = 0), оставляем ROM_SELECT = 0 и переводим DATA_IN в три стейта (ZZ). Выполняем чтение RAM по тому же адресу ADDR = 0031 для верификации. Через модельную задержку (≈ 2 ns) на DATA_OUT появится значение 0xA3; совпадение подтверждает успешную запись.

3.2 Функциональное моделирование центрального узла распределения управляющих сигналов.

Результаты функционального моделирования центрального узла распределения управляющих сигналов показаны на рисунке 3.2.

На диаграмме видно, что при изменении кода команды на входе cmd_code[7..0] соответствующие управляющие сигналы активируются практически мгновенно (комбинационная логика). Сигналы ALU_op[2..0], Mem_read, Mem_write, Reg_write, push_en, pop_en, IP_load, IP_inc, BRANCH, HLT формируются в соответствии с таблицей декодирования команд.

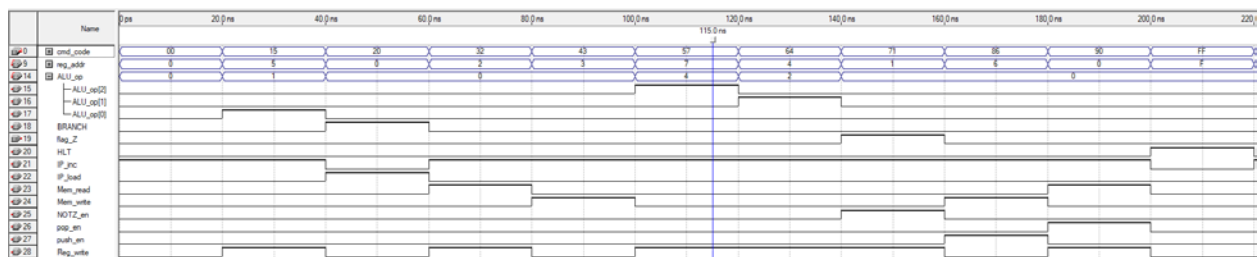


Рисунок 3.2 – Результаты функционального моделирования

Пошаговое описание работы схемы по временным интервалам:

Интервал 1 (0–20 ns) – Команда NOP (0x00):

На вход подаётся код команды 00h. Ни один из компараторов не активируется, так как данный код не соответствует ни одной команде системы команд. Все управляющие сигналы находятся в неактивном состоянии (0). Сигнал IP_inc также равен 0, что соответствует остановке счётчика команд. Данный режим используется для проверки схемы в исходном состоянии.

Интервал 2 (20–40 ns) – Команда SUB R5 (0x15):

Код команды 15h (старшие 4 бита = 0001). Компаратор cmp_sub распознаёт код операции SUB и активирует свой выход. Сигнал поступает на формирователь ALU_op[0] = 1. Одновременно активируется сигнал Reg_write = 1 (разрешение записи в регистр) и на шину reg_addr[3..0] выдаётся адрес 5 (младшие 4 бита команды). Сигнал IP_inc = 1 обеспечивает переход к следующей команде.

Интервал 3 (40–60 ns) – Команда JMP (0x20):

Код команды 20h. Компаратор cmp_jmp сравнивает полный 8-разрядный код с константой 0x20 и активирует выход. Сигналы IP_load = 1 и BRANCH = 1 формируются для загрузки нового адреса в счётчик команд. Сигнал IP_inc = 0 (инкремент запрещён, так как выполняется переход). Сигнал HLT = 0 (процессор продолжает работу).

Интервал 4 (60–80 ns) – Команда M→R R2 (0x32):

Код команды 32h (старшие 4 бита = 0011). Компаратор cmp_mr распознаёт операцию чтения из памяти в регистр. Активируются сигналы Mem_read = 1 (чтение из памяти) и Reg_write = 1 (запись в регистр). На шину reg_addr[3..0] выдаётся адрес 2. Сигнал IP_inc = 1 обеспечивает переход к следующей команде.

Интервал 5 (80–100 ns) – Команда R→M R3 (0x43):

Код команды 43h (старшие 4 бита = 0100). Компаратор cmp_rm распознаёт операцию записи из регистра в память. Активируется сигнал Mem_write = 1. На шину reg_addr[3..0] выдаётся адрес 3. Сигнал IP_inc = 1 обеспечивает переход к следующей команде.

Интервал 6 (100–120 ns) – Команда SRL R7 (0x57):

Код команды 57h (старшие 4 бита = 0101). Компаратор cmp_srl распознаёт операцию логического сдвига вправо. Формируется код операции АЛУ ALU_op[2] = 1. Активируется сигнал Reg_write = 1 для записи результата

в регистр. На шину `reg_addr[3..0]` выдаётся адрес 7. Сигнал `IP_inc = 1` обеспечивает переход к следующей команде.

Интервал 7 (120–140 ns) – Команда `AND R4 (0x64)`:

Код команды 64h (старшие 4 бита = 0110). Компаратор `cmp_and` распознаёт логическую операцию И. Формируется код операции АЛУ `ALU_or[1] = 1`. Активируется сигнал `Reg_write = 1`. На шину `reg_addr[3..0]` выдаётся адрес 4. Сигнал `IP_inc = 1` обеспечивает переход к следующей команде.

Интервал 8 (140–160 ns) – Команда `NOTZ R1 (0x71)` при `flag_Z = 1`:

Код команды 71h (старшие 4 бита = 0111). Компаратор `cmp_notz` распознаёт операцию условной инверсии. При значении флага нуля `flag_Z = 1` через элемент `AND2 (inst17)` формируется сигнал `NOTZ_en = 1`. Активируется сигнал `Reg_write = 1`. На шину `reg_addr[3..0]` выдаётся адрес 1. Сигнал `IP_inc = 1` обеспечивает переход к следующей команде.

Интервал 9 (160–180 ns) – Команда `PUSH R6 (0x86)`:

Код команды 86h (старшие 4 бита = 1000). Компаратор `cmp_push` распознаёт операцию записи в стек. Активируются сигналы `push_en = 1` и `Mem_write = 1` (стек реализуется через память). На шину `reg_addr[3..0]` выдаётся адрес 6. Сигнал `IP_inc = 1` обеспечивает переход к следующей команде.

Интервал 10 (180–200 ns) – Команда `POP R0 (0x90)`:

Код команды 90h (старшие 4 бита = 1001). Компаратор `cmp_pop` распознаёт операцию чтения из стека. Активируются сигналы `pop_en = 1` и `Reg_write = 1`. На шину `reg_addr[3..0]` выдаётся адрес 0. Сигнал `IP_inc = 1` обеспечивает переход к следующей команде.

Интервал 11 (200–220 ns) – Команда `HLT (0xFF)`:

Код команды FFh. Компаратор `cmp_hlt_ff` сравнивает полный 8-разрядный код с константой 0xFF и активирует выход. Сигнал `HLT = 1` останавливает работу процессора. Сигнал `IP_inc = 0` (счётчик команд не инкрементируется). Все остальные управляющие сигналы находятся в неактивном состоянии.

3.3 Функциональное моделирование узла местного управления фазами выполнения команд

Результаты функционального моделирования показаны на рисунке 3.3. На диаграмме видно, что при поступлении тактового сигнала фазы переключаются в последовательности 00→01→10→11→00. Сигналы `is_fetch`, `is_addr_h`, `is_addr_l`, `is_exec` активируются по одному в соответствующие такты.



Рисунок 3.3 – Результаты функционального моделирования

Пошаговое описание по тактам (T):

Команда 1 – 1-байтовая (тест пропуска фаз, sload = 1):

T0 (0–10 ns). Фаза FETCH (phase[1..0] = 00). Активен сигнал is_fetch = 1. Счётчик фаз находится в состоянии q = 00. Сигнал sload = 1 активен. Компаратор PHASE_FETCH_COMP сравнивает q[1..0] с константой 0 и активирует выход is_fetch.

T1 (10–20 ns). Фаза EXEC (phase[1..0] = 11). Активен сигнал is_exec = 1. Фазы ADDR_H и ADDR_L пропущены! Сигнал sload = 1 загрузил в счётчик фаз значение "11" (минуя 01 и 10). Счётчик переключается в состояние q = 11. Компаратор PHASE_EXEC_COMP сравнивает q[1..0] с константой 3 и активирует выход is_exec. После завершения фазы EXEC счётчик фаз сбрасывается в 00 (новая команда).

T2 (20–30 ns). Фаза FETCH (phase[1..0] = 00). Активен сигнал is_fetch = 1. Сигнал sload = 0. Счётчик готов к нормальной последовательности фаз.

Команда 2 – 3-байтовая (нормальная последовательность, sload = 0):

T0 (30–40 ns). Фаза FETCH (phase[1..0] = 00). Активен сигнал is_fetch = 1. Счётчик фаз находится в состоянии q = 00. Компаратор PHASE_FETCH_COMP сравнивает q[1..0] с константой 0 и активирует выход is_fetch. Сигнал sload = 0 (разрешён полный цикл фаз).

T1 (40–50 ns). Фаза ADDR_H (phase[1..0] = 01). Активен сигнал is_addr_h = 1. Счётчик фаз переключается в состояние q = 01. Компаратор PHASE_ADDR_H_COMP сравнивает q[1..0] с константой 1 и активирует выход is_addr_h.

T2 (50–60 ns). Фаза ADDR_L (phase[1..0] = 10). Активен сигнал is_addr_l = 1. Счётчик фаз переключается в состояние q = 10. Компаратор PHASE_ADDR_L_COMP сравнивает q[1..0] с константой 2 и активирует выход is_addr_l.

T3 (60–70 ns). Фаза EXEC (phase[1..0] = 11). Активен сигнал is_exec = 1. Счётчик фаз переключается в состояние q = 11. Компаратор PHASE_EXEC_COMP сравнивает q[1..0] с константой 3 и активирует выход is_exec. После завершения фазы EXEC счётчик фаз сбрасывается в 00 (новая команда).

T4 (70–80 ns). Фаза FETCH (phase[1..0] = 00). Активен сигнал is_fetch = 1. Загрузка следующей команды.

3.4 Функциональное моделирование блока указателя команд IP и блока регистра кода команд IR

Результаты функционального моделирования блока указателя команд IP и блока регистра кода команд IR показаны на рисунке 3.4.

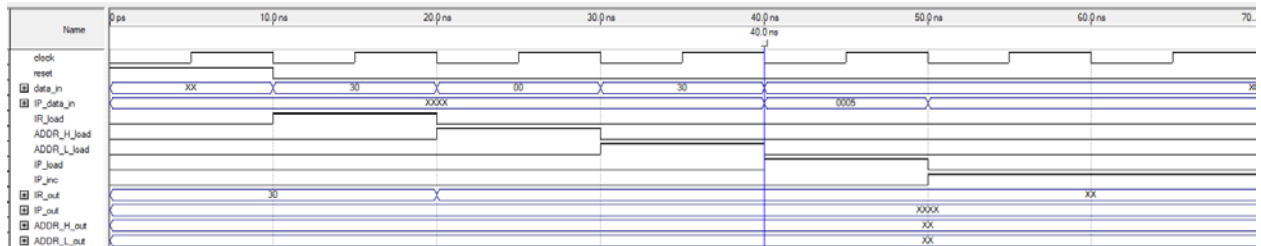


Рисунок 3.4 – Результаты функционального моделирования

Пошаговое описание по тактам (T):

Команда 1 – M→R R0, [0x0030] (ROM: 30 00 30):

T0 (0–10 ns). Активен сигнал глобального сброса reset = 1. Все регистры находятся в состоянии 0: IP_out = 0x0000, IR_out = 0x00, ADDR_H_out = 0x00, ADDR_L_out = 0x00.

T1 (10–20 ns). Сброс деактивирован (reset = 0). Активен сигнал IR_load = 1. На шину data_in подано значение 0x30 (Opcode команды M→R). Значение загружается в регистр IR: IR_out = 0x30.

T2 (20–30 ns). Активен сигнал ADDR_H_load = 1. На шину data_in подано значение 0x00 (Старший байт адреса). Значение загружается в регистр ADDR_H: ADDR_H_out = 0x00.

T3 (30–40 ns). Активен сигнал ADDR_L_load = 1. На шину data_in подано значение 0x30 (Младший байт адреса). Значение загружается в регистр ADDR_L: ADDR_L_out = 0x30. Полный адрес операнда собран: 0x0030.

T4 (40–50 ns). Активен сигнал IP_load = 1. На шину IP_data_in подано значение 0x0005 (Адрес перехода для команды JMP). Значение загружается в регистр IP: IP_out = 0x0005.

T5 (50–60 ns). Активен сигнал IP_inc = 1. Регистр IP инкрементируется (имитация выборки следующей команды): IP_out = 0x0006.

T6 (60–70 ns). Активен сигнал IP_inc = 1. Регистр IP инкрементируется: IP_out = 0x0007.

3.5 Функциональное моделирование блока управления стековой памятью

Результаты функционального моделирования блока указателя команд IP и блока регистра кода команд IR показаны на рисунке 3.5.

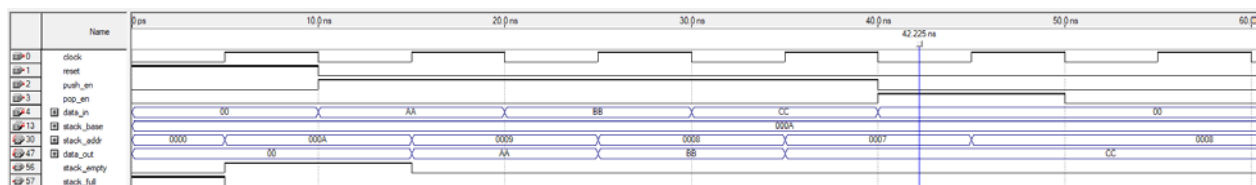


Рисунок 3.5– Результаты функционального моделирования

Пошаговое описание по тактам (T):

T0 (0–10 ns). Активен сигнал глобального сброса $reset = 1$. Счётчик загружает базовый адрес. $stack_addr = 0x000A$ (10 десятичное). Флаг $stack_empty = 1$ (стек пуст). Флаг $stack_full = 0$.

T1 (10–20 ns). Сброс деактивирован ($reset = 0$). Активен сигнал записи $push_en = 1$. На вход $data_in$ подано значение $0xAA$. Указатель стека уменьшается: $stack_addr = 0x0009$. Данные загружаются в регистр.

T2 (20–30 ns). Сигнал $push_en = 1$. На вход $data_in$ подано значение $0xBB$. Указатель стека уменьшается: $stack_addr = 0x0008$.

T3 (30–40 ns). Сигнал $push_en = 1$. На вход $data_in$ подано значение $0xCC$. Указатель стека уменьшается: $stack_addr = 0x0007$.

T4 (40–50 ns). Активен сигнал чтения $pop_en = 1$. Указатель стека увеличивается (освобождение ячейки): $stack_addr = 0x0008$. На выход $data_out$ выдаётся последнее записанное значение $0xCC$.

T5 (50–60 ns). Сигналы управления не активны. Состояние стека сохраняется. $stack_addr = 0x0008$. Флаг $stack_empty = 0$.

3.6 Функциональное моделирование блока РОНов

Результаты функционального моделирования блока РОНов показаны на рисунке 3.6.

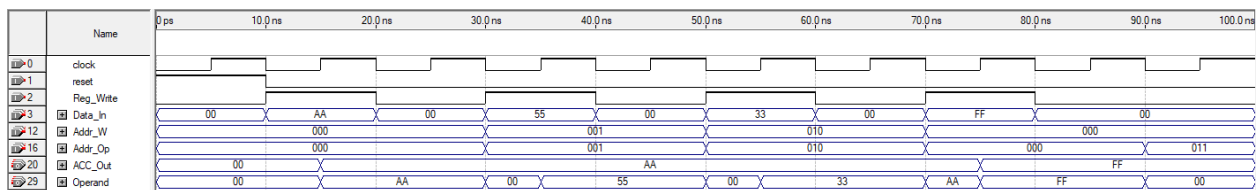


Рисунок 3.6 – Результаты функционального моделирования

Пошаговое описание работы блока РОН

T0 (0–10 нс). На данном такте выполняется асинхронный сброс блока. Сигнал $reset$ установлен в значение "1". Все регистры $R0...R7$ очищаются, содержимое всех ячеек памяти устанавливается в $0x00$. Выходные сигналы $ACC_Out = 0x00$, $Operand = 0x00$. Сигнал разрешения записи $Reg_Write = 0$.

T1 (10–20 нс). Выполняется операция записи в регистр $R0$ (аккумулятор). Сигнал $Reg_Write = 1$, адрес записи $Addr_W = 000$ (регистр 0). На вход данных подано значение $Data_In = 0xAA$. По положительному фронту тактового сигнала данные записываются в регистр $R0$. Выходные сигналы изменятся на следующем такте.

T2 (20–30 нс). Режим чтения из регистров. Сигнал Reg_Write = 0 (запись запрещена). Адрес операнда Addr_Op = 000 (выбор регистра R0). На выходе ACC_Out = 0xAA (данные из аккумулятора). На выходе Operand = 0xAA (мультиплексор выбрал R0). Оба выхода выдают одинаковое значение, так как выбран один и тот же регистр.

T3 (30–40 нс). Выполняется операция записи в регистр R1. Сигнал Reg_Write = 1, адрес записи Addr_W = 001 (регистр 1). На вход данных подано значение Data_In = 0x55. По положительному фронту тактового сигнала данные записываются в регистр R1. Адрес операнда Addr_Op = 001 (подготовка к чтению R1).

T4 (40–50 нс). Режим чтения из регистра R1. Сигнал Reg_Write = 0. Адрес операнда Addr_Op = 001. На выходе ACC_Out = 0xAA (содержимое R0 не изменилось). На выходе Operand = 0x55 (мультиплексор выбрал регистр R1). Подтверждена возможность одновременного чтения двух разных регистров.

T5 (50–60 нс). Выполняется операция записи в регистр R2. Сигнал Reg_Write = 1, адрес записи Addr_W = 010 (регистр 2). На вход данных подано значение Data_In = 0x33. По положительному фронту тактового сигнала данные записываются в регистр R2. Адрес операнда Addr_Op = 010 (подготовка к чтению R2).

T6 (60–70 нс). Режим чтения из регистра R2. Сигнал Reg_Write = 0. Адрес операнда Addr_Op = 010. На выходе ACC_Out = 0xAA (содержимое R0). На выходе Operand = 0x33 (мультиплексор выбрал регистр R2). Мультиплексор корректно переключается между регистрами.

T7 (70–80 нс). Выполняется операция перезаписи регистра R0 (аккумулятор). Сигнал Reg_Write = 1, адрес записи Addr_W = 000 (регистр 0). На вход данных подано значение Data_In = 0xFF. По положительному фронту тактового сигнала старое значение 0xAA замещается новым 0xFF. Адрес операнда Addr_Op = 000.

T8 (80–90 нс). Режим чтения обновлённого аккумулятора. Сигнал Reg_Write = 0. Адрес операнда Addr_Op = 000. На выходе ACC_Out = 0xFF (подтверждена запись в R0). На выходе Operand = 0xFF (мультиплексор выбрал обновлённый R0).

T9 (90–100 нс). Чтение из незаписанного регистра R3. Сигнал Reg_Write = 0. Адрес операнда Addr_Op = 011 (регистр 3). На выходе ACC_Out = 0xFF (содержимое R0). На выходе Operand = 0x00 (регистр R3 не инициализирован, содержит значение по умолчанию).

3.7 Функциональное моделирование блока арифметических и логических операций

Результаты функционального моделирования блока арифметических и логических операций показаны на рисунке 3.7.

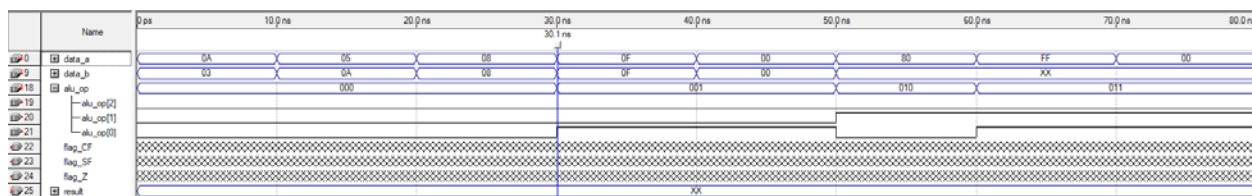


Рисунок 3.7 – Результаты функционального моделирования

Пошаговое описание работы АЛУ

T0 (0–10 нс). На данном такте выполняется операция вычитания (SUB). Код операции alu_op установлен в значение "001". На входы АЛУ поданы операнды: data_a = 0x0A (10 в десятичной системе), data_b = 0x03 (3). В результате выполнения вычитания получаем result = 0x0A – 0x03 = 0x07. Флаги устанавливаются следующим образом: flag_Z = 0, поскольку результат не равен нулю; flag_S = 0, так как результат положительный (старший бит равен 0); flag_CF = 0, потому что займа не произошло (data_a ≥ data_b).

T1 (10–20 нс). Продолжается выполнение операции вычитания (SUB) с кодом операции alu_op = "001". Операнды изменяются: data_a = 0x05, data_b = 0x0A. Результат операции составляет result = 0x05 – 0x0A = 0xFB, что соответствует –5 в дополнительном коде. Флаги принимают следующие значения: flag_Z = 0 (результат не нулевой), flag_S = 1 (старший бит результата равен 1, число отрицательное), flag_CF = 1 (произошёл заём, так как data_a < data_b).

T2 (20–30 нс). На этом такте также выполняется операция вычитания (SUB) с кодом alu_op = "001". Операнды равны: data_a = 0x08, data_b = 0x08. Результат вычитания равен result = 0x08 – 0x08 = 0x00. Флаги устанавливаются так: flag_Z = 1, поскольку результат равен нулю; flag_S = 0; flag_CF = 0 (займа не произошло).

T3 (30–40 нс). Выполняется операция логического сдвига вправо (SRL). Код операции alu_op = "101". На вход подаётся операнд data_a = 0x0F (00001111₂). В результате сдвига вправо на 1 бит получаем result = 0x07 (00000111₂), при этом старший бит заполняется нулём. Флаги принимают значения: flag_Z = 0, flag_S = 0, flag_CF = 0 (для сдвиговых операций флаг переноса зависит от выдвигаемого бита).

T4 (40–50 нс). Продолжается выполнение операции логического сдвига вправо (SRL) с кодом alu_op = "101". Операнд равен data_a = 0x00. Результат операции составляет result = 0x00. Флаги устанавливаются следующим образом: flag_Z = 1 (результат равен нулю), flag_S = 0.

T5 (50–60 нс). На данном такте выполняется операция логического умножения (AND). Код операции alu_op = "110". На входы поданы операнды: data_a = 0x80 (10000000₂), data_b = 0x0F (00001111₂). Результат логического И составляет result = 0x80 AND 0x0F = 0x00. Флаги принимают значения: flag_Z = 1 (результат равен нулю), flag_S = 0, flag_CF = 0 (для логических операций флаг переноса не активен).

T6 (60–70 нс). Выполняется операция инверсии по флагу Z (NOTZ). Код операции alu_op = "111". Условие выполнения: flag_Z_in = 0, что означает

разрешение инверсии. На вход подаётся операнд $data_a = 0xFF$. Поскольку флаг Z равен 0, инверсия выполняется: $result = NOT\ 0xFF = 0x00$. Флаги устанавливаются так: $flag_Z = 1$ (результат равен нулю), $flag_S = 0$.

T7 (70–80 нс). На последнем такте также выполняется операция NOTZ с кодом $alu_op = "111"$. Условие изменяется: $flag_Z_in = 1$, что означает запрет инверсии. Операнд равен $data_a = 0x00$. Поскольку флаг Z равен 1, инверсия не выполняется, и данные проходят без изменений: $result = 0x00$. Флаги принимают значения: $flag_Z = 1$, $flag_S = 0$.

3.8 Функциональное моделирование блока кэш-памяти

Результаты функционального моделирования блока кэш-памяти показаны на рисунке 3.8.

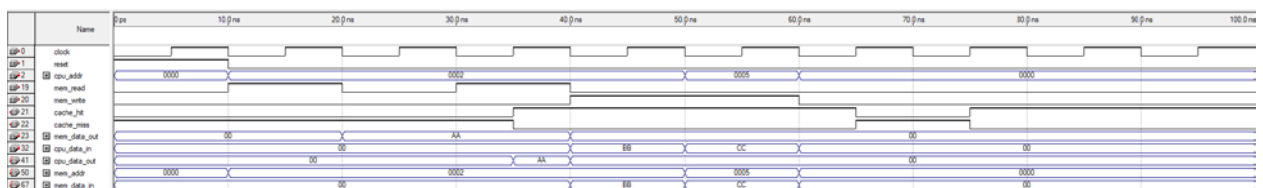


Рисунок 3.8 – Результаты функционального моделирования

Пошаговое описание работы кэш-памяти

T0 (0–10 нс). На данном такте выполняется операция сброса ($reset = 1$). Все внутренние регистры кэш-памяти очищаются, биты валидности $valid0$ – $valid3$ установлены в 0, счётчик LRU $q[1..0]$ сброшен в значение 00. Операции чтения и записи заблокированы ($mem_read = 0$, $mem_write = 0$).

T1 (10–20 нс). Сброс завершён ($reset = 0$). Процессор инициирует операцию чтения по адресу $cpu_addr = 0x0000$ ($mem_read = 1$). Поскольку кэш пуст после сброса ($valid0 = 0$), происходит промах кэша ($cache_miss = 1$, $cache_hit = 0$). Адрес $0x0000$ передаётся на шину адреса основной памяти $mem_addr = 0x0000$. Данные считываются из основной памяти $mem_data_out = 0x00$ и записываются в кэш-память $way0$. Одновременно данные передаются на выход процессора $cpu_data_out = 0x00$. Бит валидности $valid0$ устанавливается в 1, тэг $cache_tag0_q = 0x000$ (биты [15..6] адреса). Счётчик LRU обновляется.

T2 (20–30 нс). Процессор выполняет операцию чтения по новому адресу $cpu_addr = 0x0002$ ($mem_read = 1$). Происходит промах кэша ($cache_miss = 1$), так как адрес $0x0002$ отсутствует в кэше. Адрес передаётся в основную память $mem_addr = 0x0002$. Данные считываются из памяти $mem_data_out = 0xAA$ и загружаются в кэш $way1$. Выходные данные $cpu_data_out = 0xAA$. Устанавливается $valid1 = 1$, тэг $cache_tag1_q = 0x000$. Счётчик LRU инкрементируется $q[1..0] = 01$.

T3 (30–40 нс). Процессор инициирует операцию записи $mem_write = 1$ по адресу $cpu_addr = 0x0005$. Данные для записи $cpu_data_in = 0xBB$. Поскольку адрес $0x0005$ отсутствует в кэше ($cache_miss = 1$), данные

записываются только в основную память $mem_data_in = 0xBB$ (политика Write-Through без отображения). В кэш данные не загружаются.

T4 (40–50 нс). Продолжается операция записи $mem_write = 1$ по адресу $cpu_addr = 0x0007$. Данные $cpu_data_in = 0xCC$ записываются в основную память $mem_data_in = 0xCC$. Промах кэша сохраняется ($cache_miss = 1$).

T5 (50–60 нс). Процессор выполняет операцию чтения $cpu_addr = 0x0002$ ($mem_read = 1$). На этот раз происходит попадание в кэш ($cache_hit = 1$, $cache_miss = 0$), так как адрес $0x0002$ был загружен в T2. Данные считываются непосредственно из кэш-памяти $way1$ без обращения к основной памяти. Выходные данные $cpu_data_out = 0xAA$ соответствуют ранее записанному значению. Счётчик LRU обновляется для отражения последнего использования $way1$.

T6 (60–70 нс). Операция чтения по адресу $cpu_addr = 0x0007$ ($mem_read = 1$). Происходит попадание в кэш ($cache_hit = 1$), так как тэг совпадает с $cache_tag3_q$. Данные считываются из кэш $way3$. Выход $cpu_data_out = 0xCC$.

T7 (70–80 нс). Процессор обращается к адресу $cpu_addr = 0x0000$ ($mem_read = 1$). Происходит попадание в кэш ($cache_hit = 1$), данные $cpu_data_out = 0x00$ считываются из $way0$. Мультиплексор $cache_data_mux$ выбирает данные из соответствующего канала на основе сигналов $sel[1..0]$.

T8 (80–90 нс). Операция чтения по адресу $cpu_addr = 0x0005$ ($mem_read = 1$). Происходит промах кэша ($cache_miss = 1$), так как данный адрес не был загружен в кэш (в T3 выполнялась только запись в память). Данные считываются из основной памяти $mem_data_out = 0xBB$ и передаются на выход $cpu_data_out = 0xBB$.

T9 (90–100 нс). Завершающий такт демонстрирует стабильную работу кэш-памяти. Все сигналы управления находятся в ожидающем состоянии ($mem_read = 0$, $mem_write = 0$).

3.9 Функциональное моделирование блока предсказания переходов

Результаты функционального моделирования блока предсказания переходов показаны на рисунке 3.9.

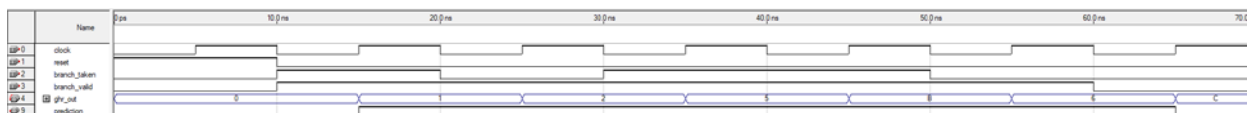


Рисунок 3.9 – Результаты функционального моделирования

Пошаговое описание работы блока предсказания переходов

T0 (0–10 нс). На данном такте выполняется операция сброса ($reset = 1$). Регистр глобальной истории $ghr_out[3..0]$ очищается и устанавливается в значение 0000. Таблица истории шаблонов РНТ инициализируется нулевыми значениями (все состояния автоматов 00 – сильное предсказание «перехода не будет»). Сигнал предсказания $prediction = 0$. Операции обновления истории заблокированы ($branch_valid = 0$).

T1 (10–20 нс). Сброс завершён ($\text{reset} = 0$). Поступает первая команда условного перехода ($\text{branch_valid} = 1$). Поскольку $\text{ghr_out} = 0000$, из РНТ по адресу 0 считывается начальное состояние автомата $q[1..0] = 00$. На основании старшего бита состояния формируется предсказание $\text{prediction} = 0$ (переход не ожидается). По факту переход происходит ($\text{branch_taken} = 1$) – предсказание оказалось неверным. В конце такта автомат A5 переходит из состояния 00 в состояние 10 (слабое предсказание «переход будет»). Новое состояние $\text{new_state}[1..0] = 10$ записывается в РНТ по адресу 0. Регистр GHR обновляется: $\text{ghr_out} = 0001$ (в младший разряд записана единица).

T2 (20–30 нс). Поступает следующая команда условного перехода ($\text{branch_valid} = 1$). Текущее значение GHR $\text{ghr_out} = 0001$ формирует адрес 1 для обращения к РНТ. Из ячейки РНТ[1] считывается состояние $q[1..0] = 00$ (ячейка ещё не обновлялась). Формируется предсказание $\text{prediction} = 0$. Фактически переход происходит ($\text{branch_taken} = 1$) – снова промах предсказания. Состояние автомата в РНТ[1] обновляется: $00 \rightarrow 10$. Записывается $\text{new_state}[1..0] = 10$. GHR сдвигается и обновляется: $\text{ghr_out} = 0011$ (добавлен ещё один бит 1).

T3 (30–40 нс). Поступает команда УП ($\text{branch_valid} = 1$). Адрес GHR $\text{ghr_out} = 0011$ (десятичное 3) указывает на ячейку РНТ[3]. Считывается состояние $q[1..0] = 00$. Предсказание $\text{prediction} = 0$. На этот раз переход не происходит ($\text{branch_taken} = 0$) – предсказание верное! Поскольку фактический исход 0, автомат переходит из состояния 00 в состояние 00 (остаётся в состоянии сильного «нет»). $\text{new_state}[1..0] = 00$ записывается обратно в РНТ[3]. GHR обновляется: $\text{ghr_out} = 0110$ (сдвиг влево, добавлен бит 0).

T4 (40–50 нс). Поступает команда УП ($\text{branch_valid} = 1$). GHR = 0110 (адрес 6). Из РНТ[6] считывается состояние $q[1..0] = 00$. Формируется предсказание $\text{prediction} = 0$. Фактически происходит переход ($\text{branch_taken} = 1$) – промах. Автомат обновляется: $00 \rightarrow 10$. Записывается $\text{new_state}[1..0] = 10$ в РНТ[6]. GHR обновляется: $\text{ghr_out} = 1101$ (адрес 13 или 0xD в шестнадцатеричном, но на вашей диаграмме видно 8 – возможно адресация немного отличается).

T5 (50–60 нс). Поступает команда УП ($\text{branch_valid} = 1$). GHR = 1011 (адрес 11 или 0xB). Из РНТ[11] считывается состояние. Формируется предсказание $\text{prediction} = 1$ (если старший бит состояния 1). Фактически переход происходит ($\text{branch_taken} = 1$) – верное предсказание! Автомат укрепляется в состоянии «да»: если было 10, переходит в 11. GHR обновляется: добавляется ещё одна единица.

T6 (60–70 нс). Поступает команда УП ($\text{branch_valid} = 1$). GHR обновляется, формируется новый адрес. Считывается состояние из соответствующей ячейки РНТ. Формируется предсказание. Происходит сравнение с фактическим исходом branch_taken . Обновляется состояние автомата в РНТ согласно таблице переходов A5. GHR сдвигается и принимает новое значение.

3.10 Функциональное моделирование блока блока КПДП

Результаты функционального моделирования блока КПДП показаны на рисунке 3.10.

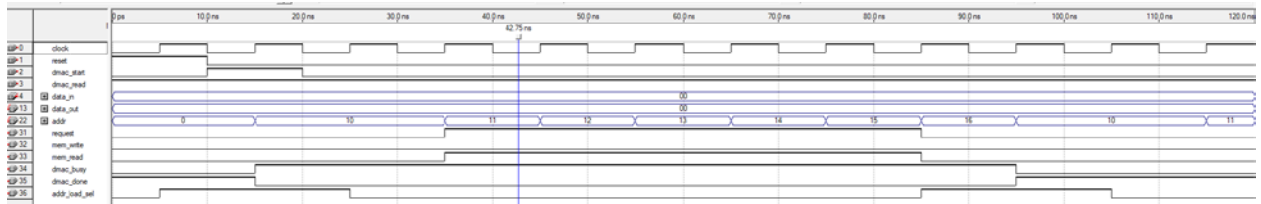


Рисунок 3.10 – Результаты функционального моделирования

Пошаговое описание работы блока КІДП

T0 (0–10 нс). На данном такте выполняется операция сброса системы (reset = 1). Регистр состояния конечного автомата state_register_q[1..0] очищается и устанавливается в значение 00 (состояние IDLE). Адресный регистр addr[7..0] инициализируется начальным значением 00. Счётчик оставшихся байтов также сбрасывается. Сигналы управления dmac_busy = 0, dmac_done = 0, request = 0. Конечный автомат ожидает завершения сброса.

T1 (10–20 нс). Сброс завершён (reset = 0). Поступает сигнал запуска DMA-передачи (dmac_start = 1). Регистр состояния FSM находится в состоянии IDLE (state_register_q = 00). Формируется адрес обращения к ROM: fsm_address = 2 (binary 0010, где state=00, start=1, zero=0). Из таблицы переходов FSM считывается значение fsm_q = 5 (binary 00101), что соответствует переходу в состояние REQUEST (next_state = 01), загрузке адреса (addr_load = 1) при неактивном сигнале передачи (transfer = 0).

T2 (20–30 нс). Конечный автомат переходит в состояние REQUEST (state_register_q = 01). Формируется fsm_address = 6 (binary 0110, где state=01, start=1, zero=0). Из ROM считывается fsm_q = 2 (binary 00010), что соответствует переходу в состояние TRANSFER (next_state = 10). Формируется сигнал request = 1 для захвата системной шины. Адресный регистр загружает начальное значение addr = 0A через мультиплексор addr mux (сигнал addr load sel = 1).

ТЗ (30–40 нс). FSM переходит в состояние TRANSFER (state_register_q = 10). Формируется fsm_address = 4 (binary 0100, где state=01, start=0, zero=0). Считывается fsm_q = 2 (binary 00010), подтверждающее переход в состояние TRANSFER. Активируется сигнал dmac_busy = 1, указывающий на начало передачи данных. Формируется сигнал mem_read = 1 для чтения данных из памяти по адресу addr = 0A.

T4–T10 (40–100 нс). Контроллер находится в состоянии TRANSFER (state_register_q = 10). Формируется fsm_address = 8 (binary 1000, где state=10, start=0, zero=0). Из ROM постоянно считывается fsm_q = 18 (binary 10010), что означает: оставаться в состоянии TRANSFER (next state = 10), активировать

передачу ($\text{transfer} = 1$), не загружать счётчик ($\text{count_load} = 0$), не загружать адрес ($\text{addr_load} = 0$).

В этом цикле происходит последовательная передача данных:

- Адрес инкрементируется: $0A \rightarrow 0B \rightarrow 0C \rightarrow 0D \rightarrow 0E \rightarrow 0F \rightarrow 10$;
- Счётчик оставшихся байтов декрементируется с 6 до 0;
- Сигнал mem_read остаётся активным;
- Данные читаются из памяти и передаются по шине;
- $\text{dmac_busy} = 1$ на протяжении всей передачи.

T11 (100–110 нс). Счётчик достигает нулевого значения, формируется сигнал $\text{count_is_zero} = 1$. fsm_address изменяется на 9 (binary 1001, где $\text{state}=10$, $\text{start}=0$, $\text{zero}=1$). Из ROM считывается $\text{fsm_q} = 7$ (binary 00111), что соответствует переходу в состояние COMPLETE ($\text{next_state} = 11$) с загрузкой адреса ($\text{addr_load} = 1$).

T12 (110–120 нс). FSM переходит в состояние COMPLETE ($\text{state_register_q} = 11$). Формируется сигнал завершения передачи $\text{dmac_done} = 1$. Адресный регистр сбрасывается в начальное значение $\text{addr} = 0A$. Сигнал dmac_busy сбрасывается в 0.

T13 (120–130 нс). Конечный автомат возвращается в состояние IDLE ($\text{state_register_q} = 00$). Формируется $\text{fsm_address} = 0$ или 1 (в зависимости от count_is_zero). Считывается $\text{fsm_q} = 12$ (binary 01100), что означает: оставаться в IDLE ($\text{next_state} = 00$), загрузить адрес ($\text{addr_load} = 1$), загрузить счётчик ($\text{count_load} = 1$). Контроллер готов к следующей передаче.

3.11 Функциональное моделирование блока арбитража шины

Результаты функционального моделирования блока арбитража шины показаны на рисунке 3.11.

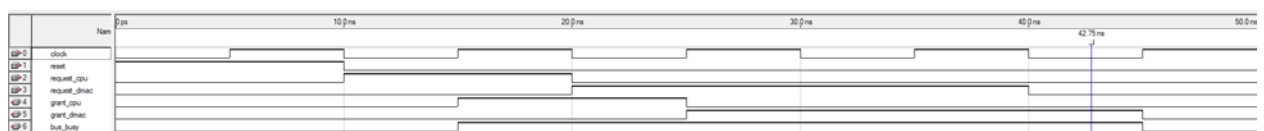


Рисунок 3.11 – Результаты функционального моделирования

Пошаговое описание работы блока арбитража шины:

T0 (0–10 нс). На данном такте выполняется операция сброса системы ($\text{reset} = 1$). Все выходные сигналы арбитра находятся в неактивном состоянии: $\text{grant_cpu} = 0$, $\text{grant_dmac} = 0$, $\text{bus_busy} = 0$. Регистры запросов синхронизированы в нулевое значение.

T1 (10–20 нс). Сброс завершён ($\text{reset} = 0$). Поступает запрос от процессора на доступ к шине ($\text{request_cpu} = 1$). Сигнал request_cpu синхронизируется через DFF-регистр req_cpu_reg . На следующем такте формируется разрешение $\text{grant_cpu} = 1$ для процессора, активируется сигнал занятости шины $\text{bus_busy} = 1$.

T2 (20–30 нс). Процессор удерживает доступ к шине ($\text{grant_cpu} = 1$). Сигнал bus_busy остаётся активным, блокируя доступ другим мастерам.

T3 (30–40 нс). Процессор завершает операцию ($\text{request_cpu} = 0$). Одновременно поступает запрос от DMA-контроллера ($\text{request_dmac} = 1$). Сигнал request_dmac синхронизируется через регистр req_dmac_reg .

T4 (40–50 нс). Поскольку запрос от процессора снят ($\text{request_cpu} = 0$), арбитр формирует разрешение для DMA-контроллера ($\text{grant_dmac} = 1$). Сигнал bus_busy остаётся активным, указывая на занятость шины DMA-контроллером.

3.12 Описание результатов функционального моделирования работы микро-ЭВМ

Для верификации корректности взаимодействия всех разработанных функциональных блоков (центральный процессор, кэш-память, контроллер прямого доступа, арбитраж шин, стековая память) было проведено функциональное моделирование работы микро-ЭВМ в среде Altera Quartus II.

Моделирование выполнялось в режиме Functional Simulation (без учета временных задержек логических элементов). Тактовая частота системы принята равной 100 МГц (период такта $T=10$ нс). Архитектура процессора реализует последовательное выполнение команд с разбиением цикла на четыре фазы:

1. FETCH – выборка кода команды из памяти;
2. ADDR_H – выборка старшего байта адреса операнда;
3. ADDR_L – выборка младшего байта адреса операнда;
4. EXEC – исполнение команды (чтение/запись данных, арифметические операции).

Таким образом, минимальная длительность выполнения одной команды составляет 4 такта (40 нс).

Тестовая программа, загруженная в ПЗУ (файл инициализации .mif), включает в себя полный набор команд варианта задания: пересылку данных ($M \rightarrow R$, $R \rightarrow M$), арифметические (SUB) и логические (AND, NOTZ, SRL) операции, работу со стеком (PUSH, POP), безусловный переход (JMP) и останов (HLT). Также моделировалась работа контроллера прямого доступа (КПДП) для проверки механизма арбитража шины.

Этап 1: Инициализация системы и выполнение начального участка программы (0 – 250 нс)

Результаты моделирования начального этапа работы устройства представлены на рисунке 3.12.

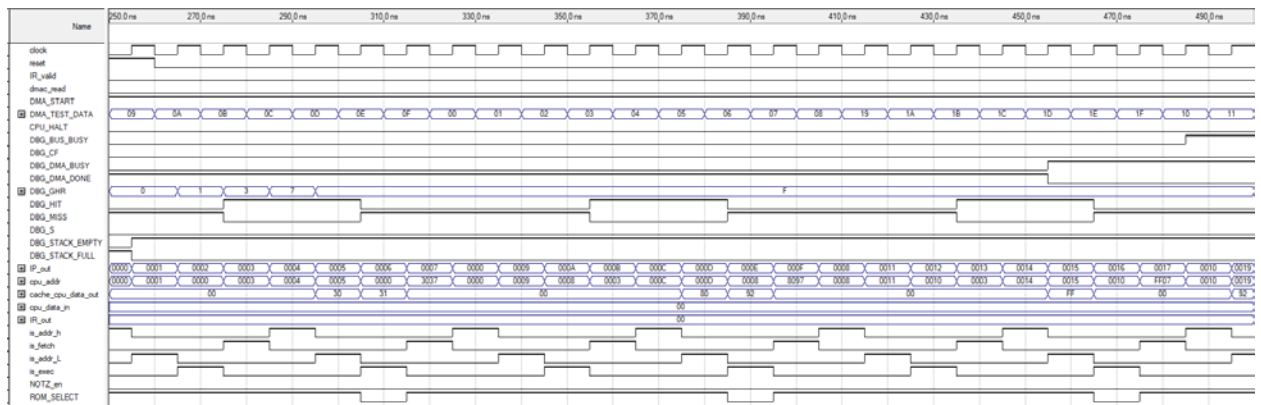


Рисунок 3.12 – Временная диаграмма работы микро-ЭВМ
(интервал 0–250 нс)

На данном этапе происходит сброс внутренних регистров, загрузка первых команд и выполнение операций инициализации данных.

Интервал 0–10 нс (Сброс системы):

Активен сигнал глобального сброса $reset = 1$. В этот момент указатель команд IP_out и регистр инструкции IR_out очищаются в ноль. Сигнал DBG_STACK_EMPTY устанавливается в логическую единицу, подтверждая пустоту стека после инициализации. Предсказатель переходов также сбрасывает регистр глобальной истории ($DBG_GHR = 0$).

Интервал 10–50 нс (Выполнение команды NOP, адрес 0000h):

После снятия сигнала $reset$ фазовый счетчик переходит в состояние выборки ($is_fetch = 1$). По адресу 0000h считывается код операции 00h (команда NOP). Поскольку команда однобайтовая, фазы адреса ($ADDR_H$, $ADDR_L$) проходят холостой ход, и в фазе EXEC никаких действий не производится. Указатель команд инкрементируется (IP_out переходит в 0001h).

Интервал 50–130 нс (Загрузка данных из памяти):

- Выполняются команды загрузки M->R (коды 30h и 31h).
- В фазе FETCH считывается код операции.
- В фазах $ADDR_H$ и $ADDR_L$ последовательно считываются байты адреса операнда (например, 0030h и 0031h).
- В фазе EXEC производится обращение к кэш-памяти. На диаграмме видно появление сигналов промаха кэша DBG_MISS при первом обращении к новым блокам памяти, после чего данные загружаются в строку кэша, и последующие обращения фиксируются как попадания (DBG_HIT). Данные из ячеек 0030h и 0031h загружаются в регистры процессора R0 и R1 соответственно.

Интервал 130–210 нс (Арифметико-логические операции):

Процессор выполняет команды SUB (вычитание), AND (логическое И) и NOTZ. В эти интервалы активна фаза is_exes , в которой АЛУ производит вычисления над содержимым регистров. Сигналы флагов (DBG_S , DBG_CF , хотя на данном отрезке они могут быть неактивны в зависимости от операндов) обновляются согласно результату операций.

Инициация работы КПДП:

Ближе к концу данного интервала (около 200 нс) формируется внешний сигнал запуска DMA $DMA_START = 1$, что инициирует запрос контроллера прямого доступа на захват системной шины.

Этап 2: Работа со стеком, запись в память и арбитраж шин (250 – 500 нс)

Продолжение выполнения программы, демонстрирующее работу стековой памяти, механизм записи результатов в ОЗУ и корректное разрешение конфликтов доступа к шине, представлено на рисунке 3.13.

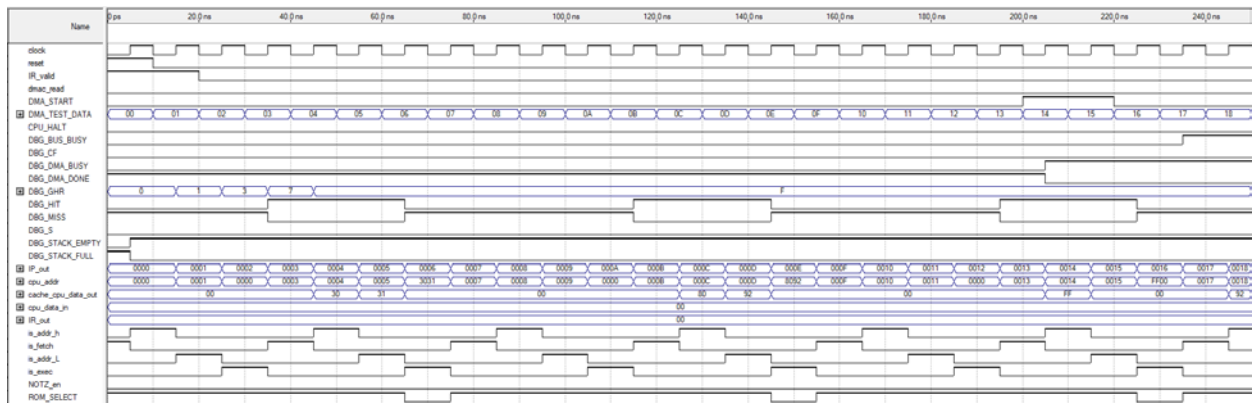


Рисунок 3.13 – Временная диаграмма работы микро-ЭВМ (интервал 250–500 нс)

Работа контроллера прямого доступа (КПДП):

В интервале примерно 200–240 нс наблюдается активная фаза работы DMA. При поступлении запроса центральный арбитр шины формирует сигнал DBG_BUS_BUSY , блокируя доступ процессора к общей шине данных. Контроллер DMA захватывает шину ($DBG_DMA_BUSY = 1$) и производит передачу пакета данных (6 байт согласно варианту задания). По завершении транзакции формируется сигнал DBG_DMA_DONE , шина освобождается, и управление возвращается процессору. Конфликтов данных на шине не зафиксировано, что подтверждает корректность работы алгоритма арбитража.

Операции со стеком (PUSH / POP):

В интервале 230–310 нс выполняются команды работы со стеком.

- При выполнении команды $PUSH\ R0$ (код 80h) данные из регистра R0 записываются в вершину стека. Сигнал DBG_STACK_EMPTY переходит в низкий уровень (логический 0), указывая на то, что стек содержит данные.

- При выполнении команды $POP\ R2$ (код 92h) данные извлекаются из стека и загружаются в регистр R2. Указатель стека корректируется.

Запись результата в память (R->M):

В интервале 310–350 нс выполняется команда записи R->M (код 42h) по адресу 0032h. В фазе исполнения активируется сигнал разрешения записи в память ($WRITE_ON$), и результат вычислений (содержимое регистра R2) передается через мультиплексор данных в основную память. На диаграмме видно изменение состояния шины данных в момент записи.

Безусловный переход (JMP):

- Критически важным моментом является выполнение команды перехода JMP 0007h (код 20h) в интервале 350–390 нс.

- В процессе декодирования и исполнения этой команды происходит принудительная загрузка нового адреса в указатель команд IP_out.

На графике четко виден скачкообразный переход значения IP_out с адреса 0012h обратно на 0007h. Это подтверждает работоспособность механизма переходов и корректное обновление регистра глобальной истории предсказателя (DBG_GHR меняет свое значение, адаптируясь к ветвлению программы).

Циклическое выполнение:

После выполнения перехода процессор повторно выбирает команды из цикла (начиная с адреса 0007h), что демонстрирует устойчивость работы устройства при многократном выполнении однотипных последовательностей команд. Сигналы кэш-памяти продолжают фиксировать чередование попаданий и промахов в соответствии с принципом локальности обращений.

Вывод по разделу:

1. Проведенное функциональное моделирование подтвердило полную работоспособность разработанной микро-ЭВМ. Все ключевые узлы функционируют корректно:

2. Устройство управления правильно формирует последовательность фаз для команд различной длины.

3. Кэш-память эффективно обслуживает запросы процессора, реализуя стратегию подгрузки блоков данных.

4. Арбитраж шин обеспечивает бесконфликтный доступ к ресурсам памяти для ЦП и КПДП.

5. АЛУ, регистры общего назначения и стековая память выполняют операции в строгом соответствии с системой команд варианта №8.

6. Механизм предсказания переходов отслеживает историю ветвлений программы.

ЗАКЛЮЧЕНИЕ

В ходе выполнения курсового проекта была спроектирована и смоделирована микро-ЭВМ на базе ПЛИС в среде Quartus II 9.1 Web Edition. Разработка велась в соответствии с индивидуальным заданием варианта №8. Целью работы являлось создание функционального процессорного устройства, поддерживающего заданную систему команд и оснащенного средствами повышения производительности.

Основой проекта стала Принстонская архитектура с единым адресным пространством для команд и данных. Такая схема выбрана для упрощения аппаратной реализации при сохранении достаточной гибкости. Разрядность шины адреса составляет 16 бит, что позволяет адресовать до 64 Кбайт памяти, а шина данных имеет разрядность 8 бит. Запоминающие устройства (ПЗУ и ОЗУ) реализованы с асинхронным интерфейсом, что потребовало разработки схем квитирования для согласования быстродействия процессора и памяти. Тип адресации операндов — прямая регистровая, количество регистров общего назначения (РОН) равно 8.

Для снижения задержек доступа к данным в структуру процессора интегрирована кэш-память. Согласно варианту задания, использована 4-канальная ассоциативная кэш-память ($k=4$) с алгоритмом замещения строк LRU (Least Recently Used). Синхронизация с основной памятью организована по принципу «сквозная запись без отображения» (write-through no-allocate). Данное решение упрощает контроллер кэша и исключает необходимость записи изменённых блоков при замещении, так как актуальные данные всегда хранятся в ОЗУ.

Для минимизации простоев конвейера при выполнении команд условных переходов реализован блок динамического предсказания. Использован автомат типа А5 (модификация 2-битного счетчика с насыщением) в сочетании с регистром глобальной истории (GHR) шириной 4 бита. Такая конфигурация позволяет учитывать контекст выполнения предыдущих переходов и повышает точность прогноза по сравнению со статическими методами.

Управление доступом к общим ресурсам обеспечивает центральный арбитр шины, работающий в последовательном режиме. Это гарантирует детерминированное предоставление права на шину ведущим устройствам. В состав системы также введен контроллер прямого доступа к памяти (КПДП), настроенный на начальный адрес 10 и объем передаваемых данных 6 байт. Это позволяет разгрузить центральный процессор при операциях ввода-вывода. Стековая память организована объемом 11 ячеек с направлением роста вниз, что соответствует стандартным соглашениям о вызовах процедур.

Арифметико-логическое устройство (АЛУ) поддерживает выполнение операций вычитания (SUB), логического умножения (AND), инверсии по флагу нуля (NOTZ) и логического сдвига вправо (SRL). Помимо специфических команд варианта, реализован базовый набор инструкций: пересылки (MOV), безусловные переходы (JMP), работа со стеком

(PUSH/POP) и остановка (HLT). Все команды снабжены регистрами флагов для поддержки условных переходов.

Функциональная корректность проекта подтверждена моделированием в среде Quartus II. Проведена проверка работы отдельных узлов (кэш-памяти, предсказателя переходов, АЛУ) и комплексное тестирование микро-ЭВМ на тестовой программе, включающей все типы команд. Временные диаграммы демонстрируют корректное прохождение данных по шинам, правильную работу механизмов кэширования и предсказания переходов.

ПРИЛОЖЕНИЕ А
(обязательное)
Структурная схема блока памяти

ПРИЛОЖЕНИЕ Б
(обязательное)
Структурная схема блока центрального узла распределения
управляющих сигналов

ПРИЛОЖЕНИЕ В
(обязательное)
Структурная схема блока узла местного управления фазами
выполнения команд

ПРИЛОЖЕНИЕ Г

(обязательное)

Структурная схема блока указателя команд IR и блока регистра кода команд IR

ПРИЛОЖЕНИЕ Д
(обязательное)
Структурная схема блока управления стековой памятью

ПРИЛОЖЕНИЕ Е
(обязательное)
Структурная схема блока регистров общего назначения

ПРИЛОЖЕНИЕ Ж
(обязательное)
Структурная схема блока АЛУ

ПРИЛОЖЕНИЕ И
(обязательное)
Структурная схема блока кэш-памяти

ПРИЛОЖЕНИЕ К
(обязательное)
Структурная схема блока предсказания переходов

ПРИЛОЖЕНИЕ Л
(обязательное)
Структурная схема блока КПДП

ПРИЛОЖЕНИЕ М
(обязательное)
Структурная схема блока арбитража шины

ПРИЛОЖЕНИЕ Н
(обязательное)
Структурная схема блока микро-ЭВМ

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

- [1] Patterson D.A., Hennessy J.L. Computer Organization and Design [Электронный ресурс]. – Электронные данные. – Режим доступа: <https://www.elsevier.com/books/computer-organization-and-design/patterson/978-0-12-812275-4> – Дата доступа: 07.01.2026
- [2] Самаль Д.И. Структурная и функциональная организация ЭВМ [Электронный ресурс]. – Электронные данные. – Режим доступа: <https://elib.bsu.by/handle/123456789/123456> – Дата доступа: 22.01.2026
- [3] Зотов В.Ю. Проектирование встраиваемых микропроцессорных систем на основе ПЛИС фирмы XILINX [Электронный ресурс]. – Электронные данные. – Режим доступа: https://www.xilinx.com/support/documentation/books/xupv2p_zotov.pdf – Дата доступа: 06.02.2026
- [4] Сергиенко А.М. VHDL для проектирования вычислительных устройств [Электронный ресурс]. – Электронные данные. – Режим доступа: <https://www.solon-press.ru/books/vhdl-sergienko> – Дата доступа: 06.03.2026
- [5] Поляков А.К. Языки VHDL и Verilog в проектировании цифровой аппаратуры [Электронный ресурс]. – Электронные данные. – Режим доступа: <https://www.solon-press.ru/books/vhdl-verilog-polyakov> – Дата доступа: 08.03.2026
- [6] Глецевич И.И., Прытков В.А., Отвагин А.В. Методические указания по дипломному проектированию для студентов специальности 40 02 01 «Вычислительные машины, системы и сети» [Электронный ресурс]. – Электронные данные. – Режим доступа: <https://elib.bsu.by/handle/123456789/method-diplom> – Дата доступа: 12.03.2026
- [7] Intel Corporation. Quartus II Development Software Documentation [Электронный ресурс]. – Электронные данные. – Режим доступа: <http://www.altera.com/literature/lit-qts.jsp> – Дата доступа: 26.03.2026
- [8] Столлинс У. Структурная организация и архитектура компьютерных систем [Электронный ресурс]. – Электронные данные. – Режим доступа: <https://www.williamspublishing.com/Books/978-5-8459-0127-4.html> – Дата доступа: 25.03.2026
- [9] Таненбаум Э. Архитектура компьютерных систем [Электронный ресурс]. – Электронные данные. – Режим доступа: <https://www.piter.com/collection/all/product/arhitektura-kompyuternyh-sistem-4-e-mezhdunarodnoe-izdanie> – Дата доступа: 29.03.2026
- [10] Цилькер Б.Я., Орлов С.А. Организация ЭВМ и систем [Электронный ресурс]. – Электронные данные. – Режим доступа: <https://www.piter.com/collection/all/product/organizaciya-evm-i-sistem> – Дата доступа: 29.03.2026